



Master Monnaie Banque Finance Assurance

Parcours Analyse des Risques Bancaires

Marchés Financiers et Théorie Financière

Réalisé par :

Mohamed RADOUAN

Christopher FRAYSSE

Ilias OUARDIRHI

Yunqing JIANG

Sous la direction de :

Mr. Jules SADEFO-KAMDEM

Année universitaire : 2024/2025

Table des matières

1	Implémentation et choix	4
1.1	Acquisition et traitement des données	4
1.1.1	Collecte des données brutes	4
1.1.2	Calcul des rendements et variance empirique	5
1.1.3	Prétraitement de la variance pour l'estimation	5
1.2	Estimation des paramètres des modèles	6
1.2.1	Paramètres de la dynamique de variance (CIR)	6
	Étape 1 :	6
	Étape 2 :	7
1.2.2	Estimation du drift historique	8
1.2.3	Estimation du coefficient de Hurst	9
1.2.4	Estimation de la corrélation prix-volatilité	9
1.3	Simulation de trajectoires	11
1.3.1	Approche numérique	11
1.3.2	Simulation du Modèle Black-Scholes	11
1.3.3	Simulation du Modèle de Heston	12
1.3.4	Simulation du Modèle MMFSV	13
1.4	Analyse de la liquidité	15
1.4.1	Mesures Basées sur le Spread	15
1.4.2	Mesures Basées sur le Volume	17
1.5	Tarification d'options par monte carlo	18
1.5.1	Choix Numériques	19
1.5.2	Spécificités par Modèle	19
2	Analyse des résultats	20
2.1	Paramètres de volatilité stochastique	20
2.1.1	Interprétation des paramètres clés	20
2.1.2	Dynamiques spécifiques par type d'actifs	21
2.1.3	Implications pour la modélisation et la gestion des risques	21
2.2	Simulation et Pricing	22
2.2.1	Méthodologie d'évaluation	22
2.2.2	Résultats de tarification des options	22
2.2.3	Résultats des simulations de trajectoires	23
2.2.4	Analyse différenciée par caractéristique de paramètres	23
2.2.5	Interprétation des divergences pour les cryptomonnaies	24
2.2.6	Implications pratiques pour la tarification et le choix de modèle	24
2.2.7	Synthèse comparative et recommandations opérationnelles	25
2.3	Proxies de liquidité	26
2.3.1	Statistiques descriptives des proxies de liquidité	26
2.3.2	Analyse des actions bancaires européennes	26

2.3.3	Spécificité des cryptomonnaies	27
2.3.4	Relation entre proxies de liquidité et performance des modèles	28
2.3.5	Implications pour la sélection de modèle et le calibrage	28
A	Annexe	30
A.1	Actions, tickers	30

Introduction

Je citerai ce qu'on raconte de Thales de Milet ; c'est une spéculation lucrative, dont on lui a fait particulièrement honneur [...]. Ses connaissances en astronomie lui avaient fait supposer, dès l'hiver, que la récolte suivante des olives serait abondante [...], il employa le peu d'argent qu'il possédait à fournir des arrhes pour la location de tous les pressoirs de Milet et de Chios ; il les eut à bon marché, en l'absence de tout autre enchérisseur. Mais quand le temps fut venu, les pressoirs étant recherchés tout à coup par une foule de cultivateurs, il les sous-loua au prix qu'il voulut. Le profit fut considérable ; et Thales prouva, par cette spéculation habile, que les philosophes, quand ils le veulent, savent aisément s'enrichir, bien que ce ne soit pas là l'objet de leurs soins.

Ainsi Aristote expliqua dans son livre Questions de Politique une spéculation réalisée par Thalès, souvent considérée comme l'une des premières options de l'Histoire : louer tous les pressoirs à olives avant la récolte, puis les sous-louer au prix fort. Contrairement aux options modernes, où le sous-jacent est souvent un actif financier, Thalès spéculait sur une ressource agricole : l'olive. Bien que le sous-jacent ait changé pour remettre au centre des options les actifs financiers, cette anecdote d'Aristote sur les raisons de l'enrichissement soudain de Thalès témoigne d'une lointaine ancienneté de cet instrument financier complexe. L'évaluation des produits dérivés financier et la modélisation des dynamiques des prix des actifs sont des domaines essentiels en finance, en particulier en finance quantitative, car elles permettent d'estimer les risques et de prévoir l'évolution des prix sur les marchés financiers.

Le modèle de Black–Scholes (1973) reste encore aujourd'hui la référence, mais ses hypothèses fortes (volatilité constante dans le temps, processus sans mémoire, "diffusion" continue) ne permettent pas de décrire la réalité d'un marché où se croisent : clusters de volatilité, corrélation négative entre le prix et la volatilité et les sauts / queues épaisses.

Le modèle d'Heston propose de faire suivre à la variance un processus à dynamique stochastique qui fait osciller la variance autour d'un niveau cible, avec une force de rappel proportionnelle à l'écart. En faisant ainsi, Heston comble les problèmes de cluster de volatilité et d'effet de levier en permettant notamment à la corrélation d'être $\rho < 0$ entre le bruit du prix et celui de la variance.

Néanmoins, supprimer l'hypothèse d'une volatilité constante n'est pas suffisant. Le modèle MMFSV va, lui, encore plus loin en comblant les deux autres limites de Black-Scholes. En reprenant le modèle d'Heston en y ajoutant un terme fractionnaire, directement dérivé du mouvement fractionnaire, dépendant d'un coefficient de Hurst, les auteurs permettent à leur modèle d'avoir une mémoire à long terme. En ajustant la corrélation entre le bruit de prix et celui de variance, avec pour résultat une covariance instantanée finie et une absence d'arbitrage, le modèle de MMFSV vient combler les deux dernières lacunes du modèle de Black-Scholes ; l'absence de mémoire et l'incapacité de produire des sauts, queues épaisses¹.

1. Le modèle MMFSV ne permet pas totalement de résoudre les problèmes de saut, il permet d'en intégrer partiellement les enjeux sans avoir à passer par un processus de poisson

Objectif de ce dossier

L'objectif de ce dossier est de confronté le modèle MMFSV à deux nouveaux marchés, la bancassurance européenne et les cryptomonnaies. Pour se faire, nous développons une implémentation python du modèle MMFSV, Heston et Black-Scholes. Nous complétons notre analyse avec l'étude des proxys de liquidités de Ayad et al 2019. Ainsi, nous testons dans des conditions réelles la valeur ajoutée du modèle MMFSV par rapport aux deux références classiques, Black-Scholes et Heston, sur six actifs hétérogènes : AXA, BNP, SG, CAGR, BTC et ETH alors que l'étude initiale portait uniquement sur les composantes du DSX ; nous étendons donc le champ d'application.

1 Implémentation et choix

Cette section vise à clarifier la démarche adoptée pour implémenter et comparer les modèles financiers Black-Scholes (BS), Heston et MMFSV. L'objectif est d'expliquer le fonctionnement du code Python développé, les traitements appliqués aux données, et les choix méthodologiques effectués. Nous avons essayé au plus possible de répliquer l'étude Djetcha & Sadefo Kamdem (2024) sur le modèle MMFSV et l'étude de Sadefo Kamdem & Ayad (2021) sur les proxys de liquidité. L'explication se veut accessible tout en permettant au lecteur de comprendre notre code sans avoir à ouvrir le fichier python.

1.1 Acquisition et traitement des données

1.1.1 Collecte des données brutes

Le point de départ est l'obtention de séries temporelles journalières pour six actifs distincts : quatre actions bancaires françaises (BNP.PA, GLE.PA, ACA.PA, CS.PA) et deux cryptomonnaies (BTC-USD, ETH-USD). La fonction `load_data_yahoo()` automatise cette acquisition via la bibliothèque `yfinance`, en récupérant le prix d'ouverture, de clôture, plus haut et plus bas journaliers ; le volume de transactions ; et les dates de cotation.

Cette fonction effectue déjà plusieurs traitements fondamentaux : gestion des variables de type (conversion des volumes en format numérique) ; harmonisation du calendrier (élimination des doublons et création d'un indice de date uniforme) ; remplissage des valeurs manquantes (*forward fill*) pour gérer les jours non cotés (weekends, jours fériés).

```

1 # --- Forward fill ---
2 # Pour Open, High, Low, Close, Volume, Adj Close : ffill est raisonnable
3 if isinstance(df_pivot.columns, pd.MultiIndex):
4     # Ffill par ticker (niveau 1) pour toutes les colonnes
5     df_pivot = df_pivot.groupby(level=1, axis=1).ffill()
6     # Repasse en format long
7     df_final = df_pivot.stack(level='Ticker', dropna=False).reset_index()
8     df_final.rename(columns={'level_0': 'Date'}, inplace=True)

```

Listing 1 – Extrait de la fonction `load_data_yahoo`

Le choix du *forward fill* comme méthode de gestion des valeurs manquantes correspond à l'hypothèse financière que la dernière cotation connue reste la meilleure estimation en l'absence

de nouvelle information, ce qui est cohérent avec la théorie de l'efficience des marchés sous sa forme faible.

1.1.2 Calcul des rendements et variance empirique

La fonction `compute_returns_and_variance()` transforme les prix bruts en mesures exploitables : calcul des log-prix $P_t = \ln(S_t)$; calcul des log-rendements $R_t = P_t - P_{t-1}$; et calcul de la variance quotidienne empirique $V_t^{daily} = R_t^2$.

```

1 def compute_returns_and_variance(df_in):
2     """
3     Ajoute log-prix, log-returns et variance QUOTIDIENNE empirique (R_t^2).
4     L'annualisation se fera plus tard en fonction du type d'actif.
5     """
6     df = df_in.copy()
7     df = df.sort_values(['Ticker', 'Date'])
8
9     df["log_price"] = np.log(df["Close"])
10    df["log_return"] = df.groupby("Ticker")["log_price"].diff()
11    # Supprimer les NaN introduits par diff()
12    df = df.dropna(subset=["log_return"])
13
14    # Calculer le carre du log-rendement (variance quotidienne non
15    # annualisee)
16    df["variance_daily_emp"] = df["log_return"] ** 2
17
18    return df.reset_index(drop=True)

```

Listing 2 – Fonction de calcul des rendements et de la variance

Un arbitrage important intervient ici concernant la fréquence annuelle : nous utilisons $f = 252$ jours ouvrés pour les actions et $f = 365$ jours calendaires pour les cryptomonnaies, conformément à leurs régimes de cotation différents. Cette distinction est cruciale pour l'annualisation correcte des rendements et des volatilités, comme mentionné explicitement dans le code :

```

1 # Constantes pour les frequences annuelles
2 FREQ_TRADITIONAL = 252
3 FREQ_CRYPTO = 365
4
5 # Dans main():
6 is_crypto = tk.endswith("-USD")
7 freq = FREQ_CRYPTO if is_crypto else FREQ_TRADITIONAL
8 dt_hist = 1.0 / freq
9 print(f" Type: {'Crypto' if is_crypto else 'Action/Trad.'}, Freq={freq}, dt
10      ={dt_hist:.5f}")

```

Listing 3 – Distinction des fréquences selon le type d'actif

1.1.3 Prétraitement de la variance pour l'estimation

La variance empirique quotidienne V_t^{daily} est d'abord annualisée en divisant par le pas de temps $dt_{hist} = 1/f$, où f est le nombre de jours de cotation par an selon le type d'actif.

Pour réduire le bruit inhérent à cette mesure, deux traitements successifs sont appliqués : winsorisation via la fonction `winsorize()` (plafonnement des valeurs extrêmes au quantile 99% pour réduire l'impact des observations aberrantes); et lissage exponentiel via la fonction `smooth_variance_exp()` (application d'une moyenne mobile exponentielle avec un paramètre $\alpha = 0.8$ donnant plus de poids aux observations récentes).

```
1 # --- Preparation variance V annualisee ---
2 V_annualized_raw = sub["variance_daily_emp"].values / dt_hist
3 variance_smooth_alpha = 0.8
4 variance_winsor_quantile = 0.99
5 V_annualized_win = winsorize(V_annualized_raw, upper_quantile=
    variance_winsor_quantile)
6 V_smooth = smooth_variance_exp(V_annualized_win, alpha=
    variance_smooth_alpha)
7 V_est = V_smooth[~np.isnan(V_smooth)]
```

Listing 4 – Pretraitement de la variance

Ces étapes permettent d'obtenir une série de variance V_t^{smooth} plus stable, servant de base fiable pour l'estimation des paramètres structurels du processus de variance.

1.2 Estimation des paramètres des modèles

1.2.1 Paramètres de la dynamique de variance (CIR)

Pour les modèles Heston et MMFSV, la variance suit un processus de Cox-Ingersoll-Ross (CIR) :

$$dV_t = \kappa(\theta - V_t)dt + \sigma\sqrt{V_t}dW_t^V \quad (1)$$

L'estimation des paramètres κ (vitesse de retour à la moyenne), θ (niveau moyen de long terme) et σ (volatilité de la volatilité) s'effectue en deux étapes, conformément à l'approche de Djetcha & Sadefo Kamdem (2024) décrite dans la section 3.5 de leur article :

Étape 1 : La fonction `estimate_olse()` implémente cette méthode initiale, basée sur la discrétisation de l'équation de variance. En référence à l'équation (45) de Djetcha & Sadefo Kamdem (2024), nous transformons :

$$\frac{V_{t+\Delta t} - V_t}{\sqrt{V_t}} = \kappa\theta\frac{\Delta t}{\sqrt{V_t}} - \kappa\Delta t\sqrt{V_t} + \sigma\eta^{\lambda,a,b,H} \quad (2)$$

Ceci correspond à une régression linéaire de la forme $Y = X\beta + \epsilon$ où : $Y = \frac{V_{t+\Delta t} - V_t}{\sqrt{V_t}}$ est la variable dépendante; $X_1 = \frac{\Delta t}{\sqrt{V_t}}$ et $X_2 = \Delta t\sqrt{V_t}$ sont les variables explicatives; $\beta_1 = \kappa\theta$ et $\beta_2 = -\kappa$ sont les coefficients à estimer.

σ est ensuite dérivé de la variance des résidus de cette régression. Cette approche fournit des estimations initiales rapides mais approximatives.

```
1 def estimate_olse(V, dt):
2     """Estimation OLSE des parametres ( , , ) du processus CIR pour V.
3     """
4     V = np.asarray(V, dtype=float)
5     valid_indices = ~np.isnan(V)
```

```

5 V = V[valid_indices] # Travailler sur les valeurs non-NaN
6
7 if len(V) < 3:
8     warnings.warn("Estimation OLSE: Pas assez de points valides.")
9     return (float(np.nan), float(np.nan), float(np.nan))
10
11 eps = 1e-8 # Pour éviter divisions par zero ou sqrt(0)
12 V_prev = np.maximum(V[:-1], eps) # Utiliser V_prev pour éviter sqrt(0)
13 V_curr = V[1:]
14
15 # Préparer les variables pour la régression: dV/sqrt(V) ~ p1/sqrt(V) +
16 # p2*sqrt(V)
17 sqrt_V_prev = np.sqrt(V_prev)
18 y = (V_curr - V_prev) / sqrt_V_prev
19 X0 = dt / sqrt_V_prev
20 X1 = dt * sqrt_V_prev
21 X = np.column_stack((X0, X1))
22
23 # Résoudre le système linéaire par moindres carrés
24 beta, residuals_sum_sq, _, _ = np.linalg.lstsq(X, y, rcond=None)
25
26 # Dédire kappa et theta
27 kappa0 = -beta[1]
28 theta0 = beta[0] / kappa0
29
30 # Estimation de sigma à partir des résidus
31 N_eff = len(y) # Nombre de points utilisés dans la régression
32 dof = N_eff - X.shape[1] # Degrés de liberté
33 resid_var = residuals_sum_sq / dof
34 sigma0 = np.sqrt(resid_var / dt) # Var(residu) sigma0^2 * dt

```

Listing 5 – Extrait de la fonction d'estimation OLSE

Étape 2 : La fonction `refine_mle()` utilise les estimations OLSE comme point de départ pour une optimisation plus précise. Nous affinons nos estimations avec un maximum de vraisemblance sur le modèle CIR, défini selon l'équation (37) de Djetcha & Sadefo Kamdem (2024) :

$$p(V_{t+\Delta t}|V_t; \theta, \Delta t) = ce^{-g-h} \left(\frac{h}{g}\right)^{\frac{q}{2}} I_q(2\sqrt{gh}) \quad (3)$$

où I_q est la fonction de Bessel modifiée de première espèce d'ordre q , et les termes c , g , h et q sont des fonctions des paramètres κ , θ et σ , détaillés dans l'équation (38) de l'étude.

La densité de probabilité conditionnelle, ou log-vraisemblance négative, est implémentée dans la fonction `neg_log_likelihood_cir` :

```

1 def neg_log_likelihood_cir(params, V, dt):
2     """Calcule l'opposé de la log-vraisemblance du processus CIR."""
3     kappa, theta, sigma = params
4     # Contraintes de base sur les paramètres
5     if not (kappa > 1e-8 and theta > 1e-8 and sigma > 1e-8):

```



```

6         return 1e12 # Penalite forte si hors domaine
7
8     eps = 1e-12 # Seuil numerique
9     V = np.asarray(V, dtype=float)
10    # Filtrer les NaN
11    V = V[~np.isnan(V)]
12    if len(V) < 2: return 1e12 # Pas assez de donnees
13
14    # Parametres de la densite de transition non-centree chi-deux
15    c = 2.0 * kappa / (sigma ** 2 * (1 - np.exp(-kappa * dt)))
16    q = 2.0 * kappa * theta / sigma ** 2 - 1.0
17    if q <= -1.0: return 1e12 # Contrainte q > -1 pour la fonction de
    Bessel
18
19    V_prev = np.maximum(V[:-1], eps) # Max avec eps pour eviter log(0)
20    V_curr = np.maximum(V[1:], eps)
21    u = c * V_prev * np.exp(-kappa * dt)
22    v = c * V_curr
23    arg = 2.0 * np.sqrt(u * v)
24
25    # Calcul de log(Iv(q, arg)) en evitant les infinis
26    with warnings.catch_warnings():
27        warnings.simplefilter("ignore", RuntimeWarning)
28        bessel_val = iv(q, arg)
29        log_bessel_term = np.log(np.maximum(bessel_val, eps))
30        inf_mask = np.isinf(bessel_val)
31        log_bessel_term[inf_mask] = arg[inf_mask] # Approximation
    logarithmique
32
33    # Calcul de la log-densite de transition
34    log_v = np.log(v)
35    log_u = np.log(u)
36    term3_ratio = 0.5 * q * (log_v - log_u)
37    logp = (np.log(c) - u - v + term3_ratio + log_bessel_term)
38
39    # Retourner l'oppose de la somme des log-densites
40    return -np.sum(logp)

```

Listing 6 – Fonction de log-vraisemblance negative pour le processus CIR

L'implémentation utilise l'algorithme L-BFGS-B, choisi pour sa capacité à gérer les contraintes de bornes sur les paramètres tout en étant efficace pour les problèmes d'optimisation de taille moyenne. Les contraintes imposées sont $\kappa > 0$, $\theta > 0$ et $\sigma > 0$, conformes aux exigences théoriques du processus CIR pour garantir la positivité de la variance.

1.2.2 Estimation du drift historique

Le paramètre μ représentant le drift (tendance) du log-prix sous la mesure historique P est calculé simplement comme la moyenne des log-rendements quotidiens, annualisée :

$$\hat{\mu} = \frac{1}{n \cdot dt_{hist}} \sum_{i=1}^n R_i \quad (4)$$

Cette estimation suppose implicitement la stationnarité des rendements, une hypothèse standard en finance malgré ses limites connues. L'implémentation dans le code est directe :

```
1 # Dans main():
2 log_returns = sub["log_return"].values
3 mu_est = np.nanmean(log_returns) / dt_hist # Drift annualise
```

Listing 7 – Estimation du drift historique

1.2.3 Estimation du coefficient de Hurst

Le coefficient de Hurst H est un paramètre crucial du modèle MMFSV, mesurant la mémoire longue d'une série temporelle. La fonction `hurst_exponent()` l'estime en utilisant l'analyse des fluctuations redressées (DFA). Cette méthode est différente de la méthode utilisée par Djautcha & Sadefo, ces derniers ont déterminé H en considérant que H était l'inverse du coefficient directeur de la droite de régression des données du marché de la DSX. Sans plus de précision sur la régression en question, nous avons décidé de nous reporter sur une méthode plus courante.

Ainsi, lors du DFA, l'algorithme décompose la série en segments de tailles croissantes, calcule une tendance polynomiale pour chaque segment, mesure les fluctuations résiduelles après suppression de cette tendance, puis détermine comment ces fluctuations évoluent avec la taille des segments. La pente de cette relation en échelle log-log fournit H .

Un choix important d'implémentation est la contrainte imposée sur H :

$$0.5 < H < 0.75 \quad (5)$$

Cette restriction, qui se traduit par $0 < \alpha < 0.25$ où $\alpha = H - 0.5$, est essentielle pour garantir l'absence d'opportunités d'arbitrage, conformément aux résultats théoriques de la section 2.2 de Djautcha & Sadefo Kamdem (2024).

```
1 # Dans main():
2 log_prices = sub["log_price"].values
3 H_est = hurst_exponent(log_prices, integrate=False) # Estime H
4 alpha_est = np.clip(H_est - 0.5, 1e-4, 0.249) # Calcul et bornage alpha
5 if H_est is not None and H_est <= 0.5:
6     warnings.warn(f"Hurst estimate ({H_est:.3f}) <= 0.5. Alpha sera clipe e
7     1e-4.")
8     alpha_est = 1e-4
```

Listing 8 – Contrainte sur le coefficient de Hurst

De plus, cela nous permet de rester cohérent avec la théorie ; si nous avons un H compris entre 0.9 et 1, nous avons de forte chance d'être face à un bruit blanc. De plus, lors de nos essais, nous avons eu à plusieurs reprises des $H > 1$ (contraire à la théorie) ce qui nous a poussés à faire un DFA sur les prix sans intégration préliminaire.

1.2.4 Estimation de la corrélation prix-volatilité

Le paramètre ρ représente la corrélation instantanée entre les innovations du prix et celles de la variance. La fonction `estimate_rho()` l'estime en calculant les résidus standardisés Z_V et Z_S des processus de variance et de prix, puis en mesurant leur corrélation empirique.

Cette approche est conforme à l'équation (34) de Djetcha & Sadefo Kamdem (2024) :

$$\langle dM_S^{H,\lambda}(t), dM_V^{H,\lambda}(t) \rangle = \rho(a + b\lambda^\alpha)^2 dt \quad (6)$$

En pratique, nous utilisons les paramètres déjà estimés ($\kappa, \theta, \sigma, \mu, \alpha$) pour extraire ces résidus standardisés, puis calculons leur corrélation de Pearson.

```

1 def estimate_rho(log_price_series, var_series, dt,
2                 kappa, theta, sigma, mu_est,
3                 a=0.9, b=1.0, lam=1e-4, alpha_frac=0.05):
4     """Estime via correlation des residus standardises Zv et Zs (MMFSV).
5     """
6
7     eps = 1e-12 # Seuil numerique
8
9     # Calculer les log-rendements
10    r = np.diff(log_price_series)
11
12    # Calculer le facteur et la serie _H
13    zeta = (a + b * lam ** alpha_frac)**2
14    eps_H_vec = eps_H_series(alpha_frac, b, lam, dt, N - 1, a)
15
16    Z_V_res, Z_S_res = [], [] # Listes pour stocker les residus Z_V et Z_S
17    # Boucle sur les intervalles [t_i, t_{i+1}]
18    for i in range(N - 1):
19        Vt = max(var_series[i], eps)
20        Vtp = max(var_series[i+1], eps)
21        sqrt_Vt = np.sqrt(Vt)
22        eps_H_i = eps_H_vec[i]
23
24        # Residu Z_V (base sur l'equation discretisee de V)
25        denom_V = sigma * sqrt_Vt * eps_H_i * sqrt_dt
26        Z_v = ((Vtp - Vt) - kappa * (theta - Vt) * dt) / denom_V
27
28        # Residu Z_S (base sur l'equation discretisee de log(S))
29        drift_S_approx = mu_est * dt
30        denom_S = sqrt_Vt * eps_H_i * sqrt_dt
31        Z_s = (r[i] - drift_S_approx) / denom_S
32
33        # Ajouter les residus valides
34        if not (np.isnan(Z_v) or np.isinf(Z_v) or np.isnan(Z_s) or np.isinf
35                (Z_s)):
36            Z_V_res.append(Z_v)
37            Z_S_res.append(Z_s)
38
39    # Calculer la correlation entre les series de residus
40    rho_est = np.corrcoef(Z_V_res, Z_S_res)[0, 1]
41
42    # Retourner la correlation estimee, bornee entre -1 et 1
43    return float(np.clip(rho_est, -0.9999, 0.9999))

```

Listing 9 – Estimation de la correlation prix-volatilite

1.3 Simulation de trajectoires

1.3.1 Approche numérique

Les trois modèles (Black-Scholes, Heston, MMFSV) sont simulés selon une approche de discrétisation de type Euler-Maruyama, choisie pour sa simplicité d'implémentation et sa robustesse. Pour un processus $dX_t = a(X_t, t)dt + b(X_t, t)dW_t$, la discrétisation d'Euler donne :

$$X_{t+\Delta t} \approx X_t + a(X_t, t)\Delta t + b(X_t, t)\sqrt{\Delta t}Z \quad (7)$$

où $Z \sim \mathcal{N}(0, 1)$ est une variable aléatoire normale standard.

1.3.2 Simulation du Modèle Black-Scholes

La fonction `simulate_black_scholes()` implémente la discrétisation du processus :

$$dS_t = S_t \mu dt + S_t \sigma dW_t \quad (8)$$

Par souci d'efficacité, nous simulons directement le logarithme du prix :

$$d \ln(S_t) = \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dW_t \quad (9)$$

La discrétisation d'Euler donne alors :

$$\ln(S_{t+\Delta t}) \approx \ln(S_t) + \left(\mu - \frac{\sigma^2}{2} \right) \Delta t + \sigma \sqrt{\Delta t} Z \quad (10)$$

Cette transformation logarithmique évite les problèmes numériques potentiels liés à l'explosion des prix et garantit la positivité des prix simulés.

```

1 def simulate_black_scholes(S0, r, sigma, T=1.0, dt=1/252):
2     """Simule Black-Scholes sous Q (drift r). Version vectorisee."""
3     N = int(T / dt) # Nombre de pas
4     S = np.empty(N + 1); S[0] = S0 # Tableau pour stocker les prix
5     sqrt_dt = np.sqrt(dt)
6     # Calcul du drift et de la diffusion pour dlnS
7     drift = (r - 0.5 * sigma ** 2) * dt
8     vol_term = sigma * sqrt_dt
9     # Generer tous les aleas normaux d'un coup
10    Z = np.random.normal(size=N)
11    # Calculer les increments de log-prix
12    dlnS = drift + vol_term * Z
13    # Calculer les log-prix par somme cumulative
14    logS = np.log(S0) + np.cumsum(dlnS)
15    # Exponentier pour obtenir les prix
16    S[1:] = np.exp(logS)
17    # Assurer un minimum (plancher numerique)
18    return np.maximum(S, 1e-10)

```

Listing 10 – Simulation du modele Black-Scholes

1.3.3 Simulation du Modèle de Heston

La fonction `simulate_heston()` implémente la discrétisation du système couplé :

$$dS_t = S_t \mu dt + S_t \sqrt{V_t} dW_t^S \quad (11)$$

$$dV_t = \kappa(\theta - V_t)dt + \sigma \sqrt{V_t} dW_t^V \quad (12)$$

$$\langle dW_t^S, dW_t^V \rangle = \rho dt \quad (13)$$

La génération de bruits corrélés s'effectue via la décomposition de Cholesky :

$$Z_S = \rho Z_V + \sqrt{1 - \rho^2} Z_2 \quad (14)$$

où Z_V et Z_2 sont des variables normales standards indépendantes.

Un point critique d'implémentation concerne la gestion de la positivité de la variance. Nous utilisons une approche de troncature simple :

$$V_{t+\Delta t} = \max \left(V_t + \kappa(\theta - V_t)\Delta t + \sigma \sqrt{\max(V_t, 0)} \sqrt{\Delta t} Z_V, 0 \right) \quad (15)$$

Ce choix, recommandé par Lord et al. (2006) dans leur étude comparative de schémas de discrétisation pour la volatilité stochastique et repris dans la section 3.4 de Djetcha & Sadefo Kamdem (2024), offre un bon compromis entre simplicité et préservation des propriétés statistiques du processus original.

```

1 def simulate_heston(S0, V0, r, kappa, theta, sigma_v, rho,
2     T=1.0, dt=1/252):
3     """Simule Heston sous Q (drift r, params V sous Q identiques P ici).
4     """
5     N = int(T / dt) # Nombre de pas
6     S = np.empty(N + 1); V = np.empty(N + 1) # Tableaux prix et variance
7     S[0], V[0] = S0, max(V0, 0) # Conditions initiales (variance >= 0)
8     sqrt_dt = np.sqrt(dt)
9     rho_comp = np.sqrt(1 - rho**2) # Terme pour decorreler les Browniens
10
11     # Generer les aleas normaux independants
12     Z1 = np.random.normal(size=N)
13     Z2 = np.random.normal(size=N)
14     # Construire les Browniens correles pour V et S
15     Z_V = Z1 # Brownien pour la variance
16     Z_S = rho * Z_V + rho_comp * Z2 # Brownien pour le prix
17
18     # Boucle de simulation pas e pas (schema d'Euler)
19     for i in range(N):
20         # Simulation de la variance V (avec troncature)
21         V_prev = max(V[i], 1e-10) # Plancher numerique
22         sqrt_V_prev = np.sqrt(V_prev)
23         drift_V = kappa * (theta - V_prev) * dt
24         diffusion_V = sigma_v * sqrt_V_prev * sqrt_dt * Z_V[i]
25         V[i+1] = max(V_prev + drift_V + diffusion_V, 0) # Schema de
        troncature

```

```

26     # Simulation du prix S (log-Euler)
27     drift_S_term = (r - 0.5 * V_prev) * dt
28     diffusion_S_term = sqrt_V_prev * sqrt_dt *
29 diffusion_S_term = sqrt_V_prev * sqrt_dt * Z_S[i]
30     # Calcul de l'increment log-prix
31     dlnS = drift_S_term + diffusion_S_term
32     # Mise a jour du prix S (exponentielle)
33     try:
34         S[i+1] = S[i] * np.exp(dlnS)
35     except OverflowError:
36         # Gerer un overflow potentiel (rare) avec une approximation
37         lineaire
38         S[i+1] = S[i] * (1 + dlnS); warnings.warn("Sim Heston:
39 Overflow.")
40     # Assurer un minimum (plancher numerique)
41     S[i+1] = max(S[i+1], 1e-10)
42
43     return S, V

```

Listing 11 – Simulation du modèle de Heston

1.3.4 Simulation du Modèle MMFSV

La fonction `simulate_mmfsv()` implémente la discrétisation du modèle complet de Djetcha & Sadefo Kamdem (2024), défini par les équations (10)-(11) de leur article :

$$dS_t = S_t \left(\mu dt + \sqrt{V_t} dM_S^{H,\lambda}(t) \right) \quad (16)$$

$$dV_t = \kappa(\theta - V_t)dt + \sigma\sqrt{V_t}dM_V^{H,\lambda}(t) \quad (17)$$

$$dM_S^{H,\lambda}(t)dM_V^{H,\lambda}(t) = \rho(a + b\lambda^\alpha)^2 dt \quad (18)$$

Où $M_S^{H,\lambda}$ et $M_V^{H,\lambda}$ sont des mouvements browniens fractionnaires modifiés, détaillés dans la section 3.1 de l'article.

L'implémentation intègre les équations (21) et (25) de Djetcha & Sadefo Kamdem (2024) pour le calcul des termes ϕ_t^λ et $\varepsilon^{H,a,b,\lambda}$:

$$\phi_t^\lambda = \alpha(t + \lambda)^{\alpha-1} \Delta B_t \quad (19)$$

$$\varepsilon^{H,a,b,\lambda} = b\alpha(t + \lambda)^{\alpha-1} \Delta t + \sqrt{\zeta^{H,a,b,\lambda}} \quad (20)$$

où $\zeta^{H,a,b,\lambda} = (a + b\lambda^\alpha)^2$.

```

1 def simulate_mmfsv(S0, V0, r_or_mu, kappa, theta, sigma_v, rho,
2                     *, a=0.9, b=1.0, lam=1e-4, alpha_frac=0.05,
3                     T=1.0, dt=1/252, risk_neutral=False):
4     """Simulation MMFSV CORRIGEE (phi eq.21, drift Q eq.14)."""
5     _check_mmfsv_domain(alpha_frac, lam) # Verifier les param7tres
6     N = int(T / dt) # Nombre de pas
7     # Initialisation des tableaux
8     S = np.empty(N+1); V = np.empty(N+1); phi = np.empty(N+1)
9     S[0], V[0], phi[0] = S0, max(V0, 0), 0.0 # Conditions initiales
10    sqrt_dt = np.sqrt(dt)

```

```

11 alpha = alpha_frac #  $H = \alpha + 0.5$ 
12 zeta = (a + b * lam**alpha)**2 #  $\{H,a,b, \}$ 
13
14 # Generer tous les aleas necessaires
15 Z_rv_v = np.random.normal(size=N) # Pour  $dW_V$ 
16 Z_rv_s_indep = np.random.normal(size=N) # Pour la partie independante
de  $dW_S$ 
17 Z_phi_std = np.random.normal(size=N) # Pour l'approximation de  $\phi_t$ 
18
19 # Precalculer les termes dependant du temps pour phi et eps_H
20 t_k = np.arange(N) * dt # Grille de temps  $t_0, \dots, t_{N-1}$ 
21 time_factor_base = np.maximum(t_k + lam, 1e-12) #  $(t_k + )$ 
22 # Prefacteur pour  $\phi_k$ :  $\cdot (t_k + )^{(-1)}$  (Eq. 21)
23 phi_prefactor = alpha * time_factor_base**(alpha - 1)
24 # Vecteur  $\text{eps}_H$ :  $b \cdot (t_k + )^{(-1)} \cdot t + \text{sqrt}( )$  (Eq. 25)
25 eps_H_vec = b * phi_prefactor * dt + np.sqrt(zeta)
26 eps_H_vec = np.maximum(eps_H_vec, 1e-12) # Assurer positivite
27
28 # Boucle de simulation
29 for k in range(N):
30     V_prev = max(V[k], 1e-10) # Variance precedente (avec plancher)
31     sqrt_V_prev = np.sqrt(V_prev)
32     eps_H_k = eps_H_vec[k] #  $\_H$  au temps  $t_k$ 
33
34     # Simuler  $\phi_{k+1}$  (approximation Eq. 21)
35     #  $\phi_k \cdot (t_k + )^{(-1)} \cdot B_k (o B_k \text{ sqrt}(t))$ 
* Z)
36     phi_k = phi_prefactor[k] * sqrt_dt * Z_phi_std[k]
37     phi[k+1] = phi_k # Stocker la valeur
38
39     # Generer les increments Browniens correles  $dW_V$  et  $dW_S$ 
40     dW_v_std = Z_rv_v[k] #  $Z_V$  standard
41     dW_s_std = rho * dW_v_std + np.sqrt(1 - rho**2) * Z_rv_s_indep[k] #
 $Z_S$ 
42
43     # Simuler  $V_{k+1}$  (Eq. 26 ou 28)
44     drift_V = kappa * (theta - V_prev) * dt
45     diffusion_V = sigma_v * sqrt_V_prev * eps_H_k * sqrt_dt * dW_v_std
46     V[k + 1] = max(V_prev + drift_V + diffusion_V, 0.0) # Troncature
47
48     # Simuler  $S_{k+1}$  (Eq. 32 ou 33)
49     # Drift depend de P ou Q
50     if risk_neutral:
51         # Drift Q (Eq. 14):  $r + b \cdot \_t \cdot \text{sqrt}(V_t) - 0.5 \cdot V_t \cdot$ 
52         drift_logS_term = (r_or_mu + b * phi_k * sqrt_V_prev - 0.5 *
V_prev * zeta)
53     else:
54         # Drift P (Eq. 12, implicite dans Eq. 29/32):
55         mu_hist = r_or_mu #  $r_{or\_mu}$  est le drift historique  $\mu$ 
56         drift_logS_term = (mu_hist + b * phi_k * sqrt_V_prev - 0.5 *
V_prev * zeta)

```

```

57
58     # Calcul de l'increment log-prix
59     dlnS = drift_logS_term * dt + sqrt_V_prev * eps_H_k * sqrt_dt *
dW_s_std
60
61     # Mise e jour du prix S
62     try:
63         S[k+1] = S[k] * np.exp(dlnS)
64     except OverflowError:
65         S[k+1] = S[k] * (1 + dlnS); warnings.warn("Sim MMFSV: Overflow
. ")
66     # Assurer plancher numerique
67     S[k+1] = max(S[k+1], 1e-10)
68
69     # Retourner les trajectoires simulees
70     return S, V, zeta, phi

```

Listing 12 – Simulation du modele MMFSV

Une particularité importante de l'implémentation est le paramètre `risk_neutral` qui permet d'effectuer la simulation soit sous la mesure historique P (avec le drift estimé $\hat{\mu}$), soit sous la mesure risque-neutre Q (avec le taux sans risque r). Cette distinction est cruciale pour différencier les simulations à des fins de validation historique (sous P) et celles pour la tarification d'options (sous Q), conformément à la théorie financière standard.

Comme illustré dans le code ci-dessus, le drift sous la mesure Q est donné par l'équation (14) de Djetcha & Sadefo Kamdem (2024) :

$$dS_t = S_t \left\{ \left(r + b\phi_t^\lambda \sqrt{V_t} \right) dt + \sqrt{V_t} \sqrt{\zeta^{H,a,b,\lambda}} dB_S(t) \right\} \quad (21)$$

Conformément à l'approche de la section 5.1 de Djetcha & Sadefo Kamdem (2024), nous avons choisi de fixer les paramètres $a = 0.9$ et $b = 1.0$ à des valeurs génériques², et $\lambda = 10^{-4}$ comme une valeur suffisamment petite pour l'approximation du processus.

1.4 Analyse de la liquidité

Cette section implémente les proxys de liquidité définis dans l'étude de Sadefo Kamdem & Ayad (2021), permettant d'analyser la liquidité des actifs sous différents angles.

1.4.1 Mesures Basées sur le Spread

Trois indicateurs principaux sont calculés :

Utilisé grâce à la fonction `calculate_roll_spread()`, la mesure Roll estime le spread bid-ask implicite à partir de l'autocovariance des variations de prix, selon l'équation (1) de Sadefo Kamdem & Ayad (2021) :

$$Cov(\Delta p_t, \Delta p_{t-1}) = -\frac{s^2}{4} \quad (22)$$

d'où $\hat{s} = 2\sqrt{-Cov(\Delta p_t, \Delta p_{t-1})}$ lorsque la covariance est négative.

2. Comme l'a confirmé Monsieur Djetcha par mail


```

1 def calculate_roll_spread(df, window=21):
2     """
3     Calcule le Roll Spread effectif base sur la covariance serielle des
4     changements de prix.
5     """
6     price_col = 'Adj Close' if 'Adj Close' in df.columns else 'Close'
7     # S'assurer que la colonne de prix est numerique
8     prices = pd.to_numeric(df[price_col], errors='coerce')
9     delta_p = prices.diff()
10    delta_p_lag = delta_p.shift(1)
11
12    # Calculer la covariance glissante entre delta_p et delta_p_lag
13    rolling_cov = delta_p.rolling(window=window).cov(delta_p_lag)
14
15    # Calculer le spread: 2 * sqrt(-cov)
16    # Gerer les cas ou la covariance est positive (violation modele ->
17    spread=0)
18    roll_spread = 2 * np.sqrt(np.maximum(0, -rolling_cov))
19
20    return roll_spread.rename("RollSpread")

```

Listing 13 – Calcul du spread de Roll

Implémentée dans la fonction `calculate_hl()`, cette mesure simple utilise l'écart relatif entre le prix le plus haut et le plus bas de la journée, selon l'équation (3) de Sadefo Kamdem & Ayad (2021) :

$$HL_t = \frac{H_t - L_t}{H_t} \quad (23)$$

Implémentée dans la fonction `calculate_corwin_schultz_spread()`, cette mesure du spread, plus sophistiquée, utilise les prix hauts et bas sur deux jours consécutifs pour estimer le spread, selon les équations (4)-(6) de Sadefo Kamdem & Ayad (2021) :

$$CS_t = (\sqrt{2} + 1) \cdot (\sqrt{\beta_t} - \sqrt{\gamma_t}) \quad (24)$$

où β_t et γ_t sont définis par :

$$\beta_t = \sum_{j=-1}^0 \ln \left(\frac{H_{t+j}}{L_{t+j}} \right)^2 \quad (25)$$

$$\gamma_t = \ln \left(\frac{\max(H_t, H_{t-1})}{\min(L_t, L_{t-1})} \right)^2 \quad (26)$$

```

1 def calculate_corwin_schultz_spread(df, *, filter_threshold=0.0005):
2     """
3     Spread Corwin-Schultz.
4     filter_threshold : ignore les jours o High (1+thr) Low .
5     """
6     high = pd.to_numeric(df['High'], errors='coerce')
7     low = pd.to_numeric(df['Low'], errors='coerce')
8

```

```

9      # filtrage HL facultatif
10     keep = high > (1 + filter_threshold) * low
11     high, low = high[keep], low[keep]
12
13     # ----- formule standard -----
14     hl_log = np.log(high / np.maximum(low, 1e-10))
15     beta   = (hl_log**2).rolling(2).sum()
16     high2  = high.rolling(2).max()
17     low2   = low.rolling(2).min()
18     gamma  = np.log(high2 / low2)**2
19
20     alpha_num = np.sqrt(2*beta) - np.sqrt(beta)
21     alpha_den = 3 - 2*np.sqrt(2)
22     alpha = (alpha_num/alpha_den) - np.sqrt(np.maximum(gamma, 0)/alpha_den)
23
24     alpha = np.clip(alpha, -10, 10)
25     spread = 2*(np.exp(alpha)-1)/(1+np.exp(alpha))
26     return pd.Series(np.maximum(spread, 0), index=high.index,
27                      name="CorwinSchultzSpread")

```

Listing 14 – Calcul du spread Corwin-Schultz

1.4.2 Mesures Basées sur le Volume

Deux indicateurs principaux sont calculés :

Implémenté dans la fonction `calculate_adv()`, l'indicateur de volume moyen quotidien calcule la moyenne mobile du volume sur une période donnée, définis par l'équation (7) de Sadefo Kamdem & Ayad (2021) :

$$ADV_{t,n} = \frac{\sum_{t-n}^t Volume_t}{n} \quad (27)$$

Un choix d'implémentation important est l'utilisation de fenêtres distinctes pour les actions (21 jours) et les cryptomonnaies (30 jours), reflétant leurs différents régimes de cotation, comme expliqué dans la section 4.5 de l'article. De cette façon, nous conservons l'analyse précédente ou nous séparons les actifs et les options à cause de leur jour de bourse différents.

```

1 def calculate_adv(df, window=21):
2     """
3     Calcule le Volume Moyen Quotidien (Average Daily Volume - ADV).
4     """
5     volume = pd.to_numeric(df['Volume'], errors='coerce').fillna(0)
6     min_periods_adv = max(1, int(window * 0.8))
7     adv = volume.rolling(window=window, min_periods=min_periods_adv).mean()
8     return adv.rename("ADV")

```

Listing 15 – Calcul du volume moyen quotidien (ADV)

Implémenté dans la fonction `calculate_lix()`, l'indicateur de liquidité LIX, développé par Danyliv et al. (2014) et repris dans l'équation (8) de Sadefo Kamdem & Ayad (2021), mesure la liquidité comme le logarithme du ratio entre la valeur échangée et l'écart haut-bas :

$$LIX_t = \log_{10} \left(\frac{Volume_t \cdot p_t}{high_t - low_t} \right) \quad (28)$$

```

1 def calculate_lix(df):
2     """
3     Calcule l'indice de liquidité LIX.
4     """
5     volume = pd.to_numeric(df['Volume'], errors='coerce')
6     close = pd.to_numeric(df['Close'], errors='coerce')
7     high = pd.to_numeric(df['High'], errors='coerce')
8     low = pd.to_numeric(df['Low'], errors='coerce')
9
10    # Eviter division par zero si High == Low
11    h_minus_l = high - low
12    h_minus_l = np.maximum(h_minus_l, 1e-9) # Remplacer 0 par un petit
    nombre
13
14    # Calculer l'argument du log10
15    value_traded = volume * close
16    lix_arg = value_traded / h_minus_l
17
18    # Calculer LIX, gerer les arguments non positifs pour log10
19    lix = np.log10(np.maximum(lix_arg, 1e-12)) # Eviter log(<=0)
20
21    return lix.rename("LIX")

```

Listing 16 – Calcul de l'indice de liquidité LIX

La fonction `calculate_liquidity_proxies()` orchestre le calcul de tous ces indicateurs pour chaque actif, en gérant les spécificités des actions et des cryptomonnaies, comme préconisé dans la section 2 de Sadefo Kamdem & Ayad (2021) qui distingue différentes dimensions de la liquidité.

1.5 Tarification d'options par monte carlo

La tarification d'options européennes est réalisée par simulation Monte Carlo, conformément au cadre théorique classique où le prix d'une option est donné par :

$$C = e^{-rT} \mathbb{E}^Q[\max(S_T - K, 0)] \quad (29)$$

Pour chaque modèle (Black-Scholes, Heston, MMFSV), les fonctions `price_option_mc_*` suivent la même logique : générer N_{paths} trajectoires du sous-jacent jusqu'à maturité T sous la mesure risque-neutre Q ; calculer le payoff de l'option pour chaque trajectoire : $\max(S_T^{(i)} - K, 0)$; moyenner ces payoffs pour estimer l'espérance ; et actualiser cette moyenne au taux sans risque r .

```

1 def price_option_mc_black_scholes(S0, r, sigma, strike, T=1.0, dt=1/252,
    n_paths=10000):
2     """Prix Call europeen MC (Black-Scholes)."""
3     payoffs = np.empty(n_paths) # Tableau pour les payoffs
4     for i in range(n_paths):
5         # Simuler une trajectoire sous Q
6         S_path = simulate_black_scholes(S0, r, sigma, T, dt)
7         # Calculer le payoff du Call e l'echeance

```

```

8     payoffs[i] = max(S_path[-1] - strike, 0)
9     # Calculer le prix actuel par actualisation de l'esperance des payoffs
10    return np.exp(-r * T) * np.mean(payoffs)

```

Listing 17 – Tarification d'options par Monte Carlo (Black-Scholes)

1.5.1 Choix Numériques

Plusieurs choix numériques importants ont été effectués : nombre de chemins $N_{paths} = 10\,000$, offrant un compromis entre précision et temps de calcul ; pas de discrétisation $\Delta t = 1/f$ où f est la fréquence annuelle adaptée au type d'actif ; taux sans risque $r = 3\%$, conforme aux conditions de marché moyennes sur la période étudiée ; et strike $K = 1.05 \times S_0$ pour les options, correspondant à des calls légèrement hors-la-monnaie.

1.5.2 Spécificités par Modèle

Pour le modèle MMFSV, la fonction `price_option_mc_mmfsv()` nécessite des considérations supplémentaires liées au mouvement brownien fractionnaire modifié. Conformément à l'équation (14) de Djetcha & Sadefo Kamdem (2024), le drift sous la mesure Q est :

$$dS_t = S_t \left\{ \left(r + b\phi_t^\lambda \sqrt{V_t} \right) dt + \sqrt{V_t} \sqrt{\zeta^{H,a,b,\lambda}} dB_S(t) \right\} \quad (30)$$

Cette complexité supplémentaire est gérée en utilisant l'approximation de ϕ_t^λ donnée par l'équation (21) et le facteur $\varepsilon^{H,a,b,\lambda}$ de l'équation (25), comme implémenté dans la fonction de simulation. De plus, nous avons considéré, comme dans l'étude, que les paramètres en P = aux paramètres en Q, ainsi nous n'estimons pas à nouveau les paramètres.

```

1 def price_option_mc_mmfsv(S0, V0, r, kappa, theta, sigma_v, rho,
2                             a, b, lam, alpha_frac,
3                             strike, T=1.0, dt=1/252, n_paths=10000):
4     """Prix Call europeen MC (MMFSV - Corrige via simulate_mmfsv)."""
5     discount_factor = np.exp(-r * T)
6     payoffs = np.empty(n_paths)
7     for i in range(n_paths):
8         # Simuler une trajectoire S sous Q (risk_neutral=True)
9         S_path, _, _, _ = simulate_mmfsv(S0, V0, r, kappa, theta, sigma_v,
10                                         rho,
11                                         a=a, b=b, lam=lam, alpha_frac=
12                                         alpha_frac,
13                                         T=T, dt=dt, risk_neutral=True)
14         # Calculer le payoff
15         payoffs[i] = max(S_path[-1] - strike, 0.0)
16     # Actualiser l'esperance
17     return discount_factor * np.mean(payoffs)

```

Listing 18 – Tarification d'options par Monte Carlo (MMFSV)

2 Analyse des résultats

2.1 Paramètres de volatilité stochastique

Notre analyse des paramètres de volatilité stochastique estimés sur les six actifs sélectionnés met en évidence des dynamiques de marché contrastées entre valeurs bancaires européennes et cryptomonnaies. Le tableau 1 synthétise les principales caractéristiques estimées par notre implémentation du modèle MMFSV.

Ticker	κ	θ	σ	ρ	H	μ
ACA.PA	19.62	0.083	1.40	-0.183	0.581	0.12
BNP.PA	20.55	0.102	1.51	-0.182	0.559	0.13
CS.PA (AXA)	22.61	0.052	1.16	-0.166	0.570	0.15
GLE.PA	21.25	0.145	1.99	-0.088	0.578	0.04
BTC-USD	38.43	0.372	4.13	0.121	0.577	0.53
ETH-USD	35.42	0.615	5.13	0.076	0.537	0.56

TABLE 1 – Paramètres MMFSV estimés (2018–2024)

2.1.1 Interprétation des paramètres clés

La vitesse de retour à la moyenne κ présente une distinction nette entre les deux classes d'actifs. Pour les valeurs bancaires, nous observons des valeurs autour de 20, ce qui correspond à une demi-vie des chocs de volatilité d'environ $\ln(2)/\kappa \simeq 0.035$ an ≈ 9 jours ouvrés ($\frac{\ln 2}{\kappa} \times 252$). Cette caractéristique suggère un marché relativement efficace où les perturbations de volatilité s'estompent en moins de deux semaines. Plus surprenant, les cryptomonnaies affichent des κ sensiblement plus élevés (35-38), impliquant une demi-vie encore plus courte, d'environ 6-7 jours calendaires. Ce résultat contre-intuitif indique que le marché des cryptomonnaies, malgré sa réputation de forte volatilité, montre une capacité remarquable à retrouver rapidement son niveau moyen de variance après un choc.

Les niveaux de volatilité de long terme θ révèlent également des écarts substantiels. Les valeurs bancaires françaises présentent des θ allant de 0,052 (AXA) à 0,145 (Société Générale), correspondant à des écarts-types annualisés entre 22% et 38%. Ces différences reflètent bien les profils de risque distincts au sein même du secteur bancaire européen, avec AXA montrant une volatilité structurellement plus faible et GLE.PA nettement plus volatile. En revanche, les cryptomonnaies affichent des θ considérablement plus élevés : 0,372 pour Bitcoin et 0,615 pour Ethereum, traduisant des écarts-types annualisés de 61% et 78% respectivement. Ces valeurs, 4 à 8 fois supérieures à celles des actions bancaires, correspondent aux attentes pour des actifs émergents à forte spéculation sur la période 2018-2024.

La volatilité de volatilité σ est particulièrement révélatrice. Le ratio $\sigma/\sqrt{\theta}$ est compris entre 5 et 8 pour les cryptomonnaies, contre environ 7 à 14 pour les actions bancaires. Cela suggère que la variance des actions, bien que plus faible en moyenne, est proportionnellement plus erratique par rapport à son niveau moyen. Toutefois, en valeur absolue, σ reste nettement plus élevé pour les cryptomonnaies (4,13 à 5,13) que pour les actions (1,16 à 1,99), confirmant

l'intuition d'une instabilité accrue sur ces marchés émergents. Au sein des bancaires, GLE.PA se distingue avec le σ le plus élevé (1,99), reflétant sa cyclicité historique plus marquée.

2.1.2 Dynamiques spécifiques par type d'actifs

Un résultat particulièrement significatif concerne la corrélation instantanée ρ entre les innovations de prix et de variance. Pour les actions bancaires, nous obtenons des valeurs négatives classiques autour de -0,18 (à l'exception de GLE.PA avec -0,09). Cette corrélation négative traduit l'effet de levier financier bien documenté sur les marchés actions : une baisse du cours entraîne généralement une hausse de la volatilité. Pour les cryptomonnaies, nous observons un phénomène opposé avec des ρ positifs (+0,12 pour BTC, +0,08 pour ETH). Bien que modestes en valeur absolue, ces coefficients sont statistiquement significatifs (t-stat environ égal à 3-4 sur plus de 1500 observations), révélant une signature distinctive : les phases de hausse des cryptomonnaies s'accompagnent d'une augmentation de la volatilité, probablement alimentée par l'afflux d'investisseurs spéculatifs et les comportements de FOMO. Cette différence fondamentale de dynamique marché-volatilité entre actions et cryptomonnaies constitue l'un des apports les plus notables de notre étude.

Concernant la mémoire longue, le coefficient de Hurst H se situe entre 0,54 et 0,58 pour l'ensemble des actifs étudiés, soit un exposant $\alpha = H - 0,5$ compris entre 0,04 et 0,08. Cette légère persistance de la volatilité justifie l'utilisation du terme fractionnaire dans notre modèle MMFSV, tout en restant bien en-deçà du seuil critique de 0,75 qui garantit l'absence d'opportunités d'arbitrage. Les valeurs obtenues sont relativement homogènes entre actions et cryptomonnaies, suggérant que le degré de mémoire longue pourrait être une caractéristique partagée par différentes classes d'actifs financiers, indépendamment de leur niveau absolu de volatilité.

2.1.3 Implications pour la modélisation et la gestion des risques

Notre estimation du drift historique μ (sous la mesure P) montre des rendements annualisés de 4% à 15% pour les actions bancaires, reflétant la période de rebond post-Covid jusqu'en 2024. Les cryptomonnaies présentent des drifts historiques beaucoup plus élevés (53% à 56%), cohérents avec la phase haussière globale observée sur cette classe d'actifs depuis 2018, malgré plusieurs corrections significatives. Il convient de rappeler que pour la tarification des options, nous utiliserons le taux sans risque r comme drift sous la mesure risque-neutre Q, conformément à la théorie financière.

D'un point de vue opérationnel, ces résultats ont des implications directes pour la modélisation et la gestion des risques. Pour les actions bancaires, la combinaison d'un θ et d'un σ modérés avec un ρ négatif suggère que les modèles de volatilité stochastique classiques comme Heston peuvent offrir une approximation raisonnable. Toutefois, la présence d'une légère mémoire longue ($H > 0,5$) justifie l'utilisation du MMFSV pour une meilleure capture des queues de distribution.

Pour les cryptomonnaies, le tableau est sensiblement différent. Les valeurs élevées de θ et σ , combinées à un ρ positif, créent une dynamique particulière où la volatilité amplifie les mouvements de prix au lieu de les contrebalancer. Dans ce contexte, le modèle MMFSV offre un avantage significatif par rapport aux approches classiques, notamment pour la tarification des options et la gestion des risques extrêmes. La corrélation positive entre prix et volatilité

souligne également un risque spécifique de sur-levier sur les positions d'options d'achat en période haussière.

En somme, nos résultats empiriques confirment non seulement la pertinence du modèle MMFSV pour capturer les dynamiques complexes de différentes classes d'actifs, mais révèlent également des signatures de marché distinctes entre valeurs bancaires européennes et crypto-monnaies. La capacité de notre modèle à s'adapter à ces caractéristiques contrastées en fait un outil précieux pour l'évaluation des produits dérivés et la gestion des risques dans des environnements de marché hétérogènes.

Ces paramètres estimés serviront de base aux simulations que nous effectuerons dans la section suivante, où nous comparerons les performances des modèles Black-Scholes, Heston et MMFSV pour la tarification d'options sur ces différents actifs.

2.2 Simulation et Pricing

Après avoir estimé les paramètres des trois modèles (Black-Scholes, Heston et MMFSV), nous avons procédé à une double analyse comparative : d'une part, la simulation de trajectoires sur une période d'un an pour évaluer la capacité des modèles à reproduire les prix historiques ; d'autre part, la tarification d'options d'achat européennes standards pour examiner comment les spécificités de chaque modèle se traduisent dans les valorisations.

2.2.1 Méthodologie d'évaluation

Pour évaluer la qualité des simulations, nous avons adopté l'erreur quadratique moyenne (Mean Squared Error, MSE) comme métrique principale, conformément à l'approche utilisée par Djetcha & Sadefo Kamdem (2024). Cette métrique quantifie l'écart entre la trajectoire simulée et la trajectoire réelle, permettant une comparaison objective entre les différents modèles. Pour chaque actif, nous avons utilisé le dernier point des données d'estimation comme valeur initiale, puis simulé une trajectoire d'un an sous la mesure historique P avec le drift estimé.

En parallèle, nous avons évalué des options d'achat européennes via la méthode de Monte Carlo sous la mesure risque-neutre Q , en utilisant un taux sans risque de 3% et un strike légèrement hors-la-monnaie ($K = 1,05 \times S_0$), conformément aux pratiques de marché standards.

2.2.2 Résultats de tarification des options

L'analyse des prix d'options générés par les trois modèles révèle des écarts significatifs de valorisation, particulièrement marqués entre le modèle MMFSV et ses alternatives. Pour les actions bancaires françaises, le modèle MMFSV produit systématiquement des primes supérieures de 30% à 50% par rapport aux modèles Black-Scholes et Heston. Ce phénomène s'explique principalement par la mémoire longue incorporée dans le MMFSV, qui accentue les queues de distribution et améliore la capture des événements extrêmes.

Pour le Crédit Agricole (ACA.PA), l'option MMFSV est valorisée à 1,83, soit environ 30% au-dessus des valorisations de Black-Scholes (1,41) et Heston (1,32). L'écart se creuse davantage pour BNP Paribas, où la prime MMFSV atteint 10,22, dépassant de plus de 50% la valorisation Heston (6,80). Une tendance similaire s'observe pour AXA (CS.PA) et Société Générale (GLE.PA), avec des primes MMFSV supérieures d'environ 45% et 36% respectivement.

Ces écarts substantiels reflètent la capacité du modèle MMFSV à intégrer deux effets complémentaires : l'effet de levier négatif ($\rho < 0$) qui augmente la volatilité en période baissière, et la composante fractionnaire qui amortit progressivement les chocs de variance tout en maintenant une mémoire longue. Pour les investisseurs, ces valorisations plus élevées traduisent le coût supplémentaire de l'assurance contre les événements extrêmes que le MMFSV capture mieux que ses concurrents.

Le contraste est encore plus saisissant pour les cryptomonnaies. Le modèle MMFSV valorise l'option sur Bitcoin à 33 626,5, soit 55% au-dessus de la valorisation Heston (22 181,9) et Black-Scholes (22 022,8). Pour Ethereum, l'écart atteint même 75%, avec une prime MMFSV de 1 743,3 contre 994,9 pour Heston. Toutefois, comme nous le verrons dans l'analyse des trajectoires, ces écarts considérables soulèvent des questions sur la capacité du MMFSV à modéliser adéquatement les actifs caractérisés par une volatilité extrêmement élevée couplée à une corrélation prix-volatilité positive.

2.2.3 Résultats des simulations de trajectoires

Notre analyse des trajectoires simulées révèle des performances contrastées selon les classes d'actifs et la structure des paramètres estimés. Pour les actions bancaires françaises, à l'exception notable du Crédit Agricole, le modèle MMFSV s'avère généralement supérieur, capturant mieux les dynamiques de prix observées sur la période d'étude.

Pour BNP Paribas, le MSE du MMFSV (56,41) est près de 2,2 fois inférieur à celui de Black-Scholes (123,1) et 5,5 fois inférieur à celui de Heston (308,3). La performance du MMFSV est encore plus remarquable pour AXA, où son MSE de 6,75 représente une amélioration d'un facteur 7 par rapport à Black-Scholes (46,89) et d'un facteur 13 par rapport à Heston (89,10). Pour Société Générale, le MMFSV (MSE de 52,66) conserve un léger avantage sur Heston (62,88), tous deux surpassant nettement Black-Scholes (273,2).

Le Crédit Agricole constitue une exception notable où le modèle Black-Scholes fournit une meilleure approximation, avec un MSE d'environ 3,2 contre 7,3 pour le MMFSV et 28 pour Heston. Cette particularité s'explique probablement par la structure paramétrique spécifique d'ACA.PA, caractérisée par un ratio $\frac{\sigma}{\sqrt{\theta}}$ relativement faible, induisant une dynamique proche d'une log-normale simple.

À l'inverse, pour les cryptomonnaies (Bitcoin et Ethereum), le modèle de Heston surpasse nettement les autres approches. Les MSE obtenus avec Heston ($0,20310^9$ pour BTC, $0,61810^6$ pour ETH) sont approximativement un ordre de grandeur inférieurs à ceux du modèle Black-Scholes, et près de 30 fois inférieurs à ceux du MMFSV. Ce résultat, qui peut sembler contre-intuitif au regard de la complexité supposée supérieure du MMFSV, mérite une analyse approfondie des mécanismes sous-jacents.

2.2.4 Analyse différenciée par caractéristique de paramètres

Au-delà de la simple distinction par classe d'actifs, nos résultats suggèrent que c'est la signature paramétrique qui détermine véritablement la pertinence relative des modèles. Trois configurations principales émergent de notre analyse, conformément aux observations de Sadefo Kamdem & Ayad (2021) sur la relation entre paramètres estimés et performances des modèles :

Pour les actifs caractérisés par une volatilité de volatilité modérée ($\sigma \lesssim 2$) et un ratio $\sigma/\sqrt{\theta} < 10$, comme ACA.PA, les excursions de variance restent suffisamment limitées pour que la dynamique des prix se rapproche d'une log-normale. Dans ce contexte, le modèle Black-Scholes peut s'avérer suffisant, malgré sa simplicité conceptuelle.

Lorsque σ est du même ordre de grandeur que θ , mais avec une corrélation négative ($\rho < 0$) et un exposant fractionnaire modéré ($0,05 \leq \alpha \leq 0,08$), le modèle MMFSV offre un avantage significatif. Cette configuration, observée pour BNP.PA, CS.PA et GLE.PA, correspond à un effet de levier négatif amorti par la composante fractionnaire. La volatilité augmente lors des phases baissières mais redescend progressivement, créant une asymétrie que le MMFSV capture efficacement, ce qui explique également les primes d'options plus élevées qu'il génère.

Enfin, dans les situations où $\sigma \gg \theta$ et $\kappa \gg 20$, particulièrement lorsque $\rho \geq 0$, comme pour BTC-USD et ETH-USD, le modèle de Heston démontre une plus grande robustesse. Dans ce régime caractérisé par des "bouffées" de variance violentes et une absence d'effet de levier négatif, le MMFSV tend à amplifier excessivement les fluctuations de prix, générant des trajectoires qui divergent fortement du réalisé. Cette même sur-amplification explique les primes d'options considérablement plus élevées produites par le MMFSV pour les cryptomonnaies, suggérant une surévaluation potentielle du risque pour cette classe d'actifs.

2.2.5 Interprétation des divergences pour les cryptomonnaies

La contre-performance notable du MMFSV sur les cryptomonnaies mérite une attention particulière. Conformément aux observations de Sadefo Kamdem & Ayad (2021) sur les marchés en période de stress, nous constatons que la combinaison de paramètres spécifiques aux cryptomonnaies crée une dynamique particulièrement instable dans le cadre du MMFSV.

Spécifiquement, la conjonction d'une volatilité de volatilité très élevée ($\sigma > 4$), d'une vitesse de retour à la moyenne importante ($\kappa > 35$), d'une corrélation légèrement positive ($\rho > 0$) et d'un exposant fractionnaire faible ($\alpha \approx 0,04 - 0,08$) conduit à une amplification excessive des mouvements de prix. Dans cette configuration, le terme $\varepsilon^{H,a,b,\lambda}$ de l'équation (25) de Djetcha & Sadefo Kamdem (2024) se trouve dominé par $\sqrt{\zeta}$, réduisant l'effet d'amortissement que la composante fractionnaire devrait apporter.

Cette amplification se manifeste par des pics de variance qui génèrent des spikes de prix, parfois deux à trois fois supérieurs aux valeurs observées. Un examen des trajectoires simulées révèle que les flash-crashes ou rallies présents dans les données historiques se trouvent considérablement exagérés dans les simulations MMFSV, entraînant une divergence progressive vis-à-vis du réalisé.

En revanche, le modèle de Heston, grâce à sa structure plus simple et à l'absence du terme fractionnaire potentiellement déstabilisant, maintient une volatilité plus contrôlée. Sa capacité à gérer la vol-de-vol élevée et la corrélation positive estimée lui permet d'épouser les fluctuations du marché des cryptomonnaies sans générer d'explosions injustifiées.

2.2.6 Implications pratiques pour la tarification et le choix de modèle

Ces résultats suggèrent une heuristique pragmatique pour la sélection ex-ante du modèle le plus approprié, tant pour la simulation de trajectoires que pour la tarification d'options.

Pour un nouvel actif, une première passe d'estimation permet d'évaluer la signature paramétrique (σ, ρ, α) . Si $\sigma < 2$, $\rho \approx 0$ et $\sigma\sqrt{\theta} < 10$, un modèle Black-Scholes perturbé peut s'avérer suffisant. Cette configuration correspond aux actifs à volatilité relativement stable, comme illustré par le cas d'ACA.PA.

Si cette première condition n'est pas remplie, la valeur de ρ fournit une orientation cruciale. Une corrélation négative ($\rho < 0$) suggère de privilégier le MMFSV pour la tarification d'options, sa capacité à intégrer l'effet de levier et la mémoire longue produisant des valorisations plus conservatrices qui reflètent mieux le risque de queue. Les primes plus élevées générées par le MMFSV sur les actions bancaires (30-50% au-dessus de Heston) peuvent être interprétées comme le coût d'une couverture plus complète contre les événements extrêmes.

À l'inverse, une corrélation positive ($\rho \geq 0$) ou un ratio $\frac{\sigma}{\theta}$ très élevé ($\sigma > 3\theta$) indique une préférence pour le modèle de Heston, tant pour la simulation que pour la tarification. Dans ce cas, les primes d'options générées par le MMFSV risquent d'être excessivement conservatrices, surchargeant potentiellement le coût de la couverture. Ce phénomène est particulièrement visible pour les cryptomonnaies, où les primes MMFSV dépassent de 55-75% celles de Heston.

Concernant spécifiquement le paramètre de Hurst, si l'estimation donne $H < 0,53$ (soit $\alpha < 0,03$), il peut être judicieux de forcer $\alpha = 0$, ce qui revient à utiliser un modèle de Heston standard, évitant ainsi le sur-amplificateur fractionnaire observé sur les cryptomonnaies.

2.2.7 Synthèse comparative et recommandations opérationnelles

En définitive, notre analyse comparative des trajectoires simulées et des prix d'options révèle que le modèle "optimal" n'est pas intrinsèquement lié à la classe d'actifs, mais plutôt à une signature paramétrique spécifique, principalement caractérisée par le triplet (σ, ρ, α) .

Pour les actions bancaires européennes (à l'exception d'ACA.PA), le MMFSV s'impose comme le modèle de référence³, tant pour la simulation que pour la tarification. Sa capacité à intégrer la mémoire longue et l'effet de levier négatif lui permet de reproduire des trajectoires plus fidèles au réalisé et de générer des primes d'options intégrant mieux le risque de queue. Les valorisations plus élevées qu'il produit, bien que potentiellement coûteuses pour les acheteurs d'options, reflètent une évaluation plus prudente du risque, particulièrement pertinente dans un contexte de gestion de portefeuille institutionnel.

Pour les cryptomonnaies, le modèle de Heston représente un compromis plus équilibré. Ses simulations épousent mieux la dynamique observée des prix, évitant les amplifications excessives générées par le MMFSV. Ses valorisations d'options, plus modérées, semblent mieux adaptées à ces marchés où la corrélation prix-volatilité positive crée une dynamique particulière, différente de l'effet de levier classique des actions.

Le modèle Black-Scholes conserve une pertinence opérationnelle pour les titres très liquides à faible instabilité de variance, comme illustré par le cas d'ACA.PA. Toutefois, sa validité se limite à ce créneau spécifique, soulignant l'importance des modèles à volatilité stochastique pour la majorité des applications financières contemporaines.

3. En plus des commentaires fait dans les parties antérieurs (2.2.2, 2.2.4, 2.2.6, 2.2.7) nous estimons que cette partie répond à la question sur les cas où le modèle MMFSV performe le mieux

Ces observations, conformes aux travaux de Djeutcha & Sadefo Kamdem (2024) et de Sadefo Kamdem & Ayad (2021), confirment l'intérêt d'une approche adaptative dans la modélisation financière. La structure fractionnaire apporte une valeur ajoutée significative pour les actifs "moyennement turbulents" caractérisés par un effet de levier négatif, tandis qu'un modèle de Heston bien calibré demeure plus robuste face aux actifs hypervariables présentant une corrélation prix-volatilité positive.

2.3 Proxies de liquidité

Complétant notre analyse des modèles de volatilité stochastique, cette section examine les caractéristiques de liquidité des actifs étudiés.⁴ Conformément à la méthodologie de Sadefo Kamdem & Ayad (2021), nous avons calculé cinq proxies de liquidité distincts permettant d'évaluer différentes dimensions de la liquidité de marché : la largeur (spread), la profondeur (volume) et l'ampleur (breadth).

2.3.1 Statistiques descriptives des proxies de liquidité

Le tableau 2⁵ présente les statistiques descriptives des cinq proxies de liquidité calculés pour chaque actif. Ces mesures révèlent des différences structurelles significatives entre les marchés d'actions traditionnels et les cryptomonnaies.

Ticker	RollSpread moy / é-t	CS-Spread moy / é-t	LIX moy / é-t	LogVol moy / é-t	ADV moy / é-t
ACA.PA	0,100 / 0,117	0,00524 / 0,00875	8,455 / 0,178	15,571 / 0,469	6,52M / 2,44M
BNP.PA	0,540 / 0,606	0,00552 / 0,00821	8,177 / 0,163	15,020 / 0,486	3,77M / 1,46M
CS.PA	0,188 / 0,230	0,00424 / 0,00693	8,465 / 0,181	15,397 / 0,503	5,57M / 2,47M
GLE.PA	0,304 / 0,362	0,00616 / 0,00996	8,194 / 0,157	15,177 / 0,512	4,51M / 1,76M
BTC-USD	792,25 / 1077,58	0,00975 / 0,01367	11,872 / 0,236	23,966 / 0,587	30,01G / 14,35G
ETH-USD	51,84 / 79,84	0,01279 / 0,01856	11,432 / 0,234	23,219 / 0,656	14,82G / 8,51G

TABLE 2 – Statistiques descriptives des proxies de liquidité (2018–2024)

Les mesures basées sur le spread (RollSpread et CS-Spread) quantifient la largeur du marché et les coûts de transaction implicites. Le LIX combine volume, prix et amplitude pour fournir un indice global de liquidité. Les mesures basées sur le volume (LogVol et ADV) capturent la profondeur du marché, indiquant combien de titres peuvent être échangés sans impact significatif sur le prix.

2.3.2 Analyse des actions bancaires européennes

Pour les actions bancaires françaises (ACA.PA, BNP.PA, CS.PA, GLE.PA), l'analyse des proxies de liquidité révèle un marché globalement efficient et liquide, avec néanmoins des nuances entre les titres.

Le RollSpread moyen varie de 0,100 (ACA.PA) à 0,540 (BNP.PA), avec des écarts-types modérés, indiquant des coûts de transaction implicites contenus et relativement stables. Cette

4. Les parties 2.3.2 et 2.3.3 répondent à la question 5.2

5. Se référer au code .csv téléchargé par le code

faible dispersion du spread, particulièrement pour ACA.PA et CS.PA, suggère des marchés profonds, où les ordres d'achat et de vente se rencontrent sans friction excessive. Le CS-Spread confirme cette tendance avec des valeurs très resserrées, comprises entre 0,00424 et 0,00616, similaires pour l'ensemble des valeurs bancaires, traduisant une grande homogénéité des coûts de transaction explicites.

L'indice global de liquidité LIX, compris entre 8,177 et 8,465 avec une faible dispersion ($\sigma \approx 0,16 - 0,18$), reflète une bonne capacité de ces marchés à absorber des volumes substantiels avec un impact limité sur l'amplitude des prix. Cette caractéristique, que Sadefo Kamdem & Ayad (2021) qualifient de "breadth" (ampleur), est particulièrement favorable aux grandes transactions institutionnelles.

Les mesures volumétriques confirment la liquidité substantielle de ces titres. Le volume quotidien moyen (ADV) varie de 3,77 millions (BNP.PA) à 6,52 millions (ACA.PA) de titres, avec des écarts-types relativement contenus, témoignant de la profondeur et de la régularité des échanges. Cette stabilité des volumes, attestée également par le LogVol ($\approx 15 - 15,6$), suggère peu d'épisodes de "crises de liquidité", à l'exception notable de la période de mars 2020 (choc COVID-19).

Ces caractéristiques de liquidité élevée et stable expliquent en partie pourquoi, comme nous l'avons constaté précédemment, le modèle Black-Scholes peut parfois suffire pour des actions comme ACA.PA. En effet, Sadefo Kamdem & Ayad (2021) démontrent qu'un marché à faible spread et forte profondeur permet une tarification quasi sans friction, où la complexité additionnelle des modèles de volatilité stochastique n'apporte un avantage significatif qu'en période de stress marqué.

2.3.3 Spécificité des cryptomonnaies

Le contraste est saisissant lorsqu'on examine les proxies de liquidité des cryptomonnaies (BTC-USD, ETH-USD), révélant une structure de marché fondamentalement différente.⁶

Le RollSpread moyen atteint des valeurs extraordinairement élevées : 792,25 pour Bitcoin et 51,84 pour Ethereum, avec des écarts-types (respectivement 1077,58 et 79,84) témoignant d'une extrême volatilité des coûts de transaction implicites. Ces valeurs, plusieurs ordres de grandeur au-dessus de celles des actions bancaires, traduisent des phases récurrentes de stress liquidatif où les spreads s'élargissent considérablement. Le CS-Spread, bien que moins spectaculaire (0,00975 pour BTC, 0,01279 pour ETH), reste environ deux fois supérieur à celui des actions, confirmant des coûts transactionnels structurellement plus importants.

Paradoxalement, l'indice LIX présente des valeurs exceptionnellement élevées (11,872 pour BTC, 11,432 pour ETH), nettement supérieures à celles des actions bancaires. Cette apparente contradiction s'explique par les volumes colossaux échangés quotidiennement, comme le confirment les mesures LogVol ($\approx 23,2 - 24,0$) et ADV (14,82 à 30,01 milliards de dollars). Ces volumes astronomiques permettent d'absorber des transactions importantes, mais la grande dispersion de l'ADV (écarts-types de 8,51 à 14,35 milliards) révèle une profondeur hautement variable.

6. Nous abordons en particulier les cryptomonnaies pour répondre à la question 5.3. Ces dernières étant un excellent cas d'étude pour répondre à cette question (différence prononcée)

Cette configuration particulière — grande "breadth" (LIX élevé) mais "width" très instable (RollSpread extrême et volatile) — correspond au profil des marchés sujets à des crises de liquidité localisées que Sadefo Kamdem & Ayad (2021) qualifient de "stress exogène amplifié". Dans ces conditions, conformément à nos observations précédentes, le modèle de Heston s'avère le plus robuste car il capture la volatilité explosive sans l'amplifier excessivement, contrairement au modèle MMFSV qui, avec son amortisseur fractionnaire, peut sous-estimer les pics de volatilité lorsque α est faible par rapport à σ .

2.3.4 Relation entre proxies de liquidité et performance des modèles

L'analyse croisée des caractéristiques de liquidité et des performances de simulation et de tarification révèle des alignements significatifs entre les "signatures" de liquidité et les modèles optimaux.

Pour les actifs caractérisés par un RollSpread faible ($< 0,2$) et un CS-Spread homogène ($\approx 0,005$), comme ACA.PA et dans une moindre mesure CS.PA, nous observons que la dynamique des prix se rapproche d'une log-normale, avec une volatilité σ modérée et une corrélation ρ proche de zéro. Dans ce contexte de marché quasi frictionless, le modèle Black-Scholes offre souvent une approximation suffisante, conformément à nos résultats de simulation où ACA.PA présentait le MSE le plus faible sous Black-Scholes.

Les actions présentant un RollSpread plus élevé ($0,3-0,6$) et une légère dispersion des spreads, comme BNP.PA et GLE.PA, correspondent précisément aux cas où nous avons observé un effet de levier négatif ($\rho < 0$) plus marqué et où le modèle MMFSV surpasse significativement ses concurrents. Ces marchés, caractérisés par des frictions modérées mais régulières, bénéficient particulièrement de la mémoire longue incorporée dans le MMFSV, qui capture l'amortissement progressif des chocs de volatilité.

Enfin, les cryptomonnaies, avec leur RollSpread extrêmement élevé et volatile (> 50) et leur CS-Spread supérieur ($> 0,01$), constituent un cas à part. Leur signature de liquidité — volume colossal mais hautement variable, spreads explosifs — crée un environnement où la volatilité peut s'auto-entretenir via des spirales de liquidité. Dans ce contexte, le modèle de Heston, qui ne présume pas d'amortissement fractionnaire spécifique, s'avère plus robuste que le MMFSV pour capturer ces dynamiques extrêmes, comme l'attestent nos résultats de simulation où l'erreur quadratique moyenne était significativement plus faible sous Heston pour BTC-USD et ETH-USD.

2.3.5 Implications pour la sélection de modèle et le calibrage

Cette analyse des proxies de liquidité enrichit considérablement notre heuristique de sélection de modèle. Au-delà des paramètres intrinsèques de volatilité stochastique (σ, ρ, α), les caractéristiques de liquidité fournissent un indicateur avancé de la structure de marché sous-jacente.

Pour les nouveaux actifs ou en phase de pré-analyse, un simple examen du RollSpread et du LIX peut orienter efficacement le choix du modèle. Un RollSpread moyen inférieur à $0,2$ avec un LIX stable autour de $8,5$ suggère un marché où Black-Scholes peut constituer une première approximation raisonnable. Un RollSpread compris entre $0,3$ et $0,6$ avec un LIX légèrement plus

variable indique un contexte favorable au MMFSV, particulièrement si une analyse préliminaire révèle un $\rho < 0$.

Pour les marchés à liquidité extrêmement variable, caractérisés par un RollSpread > 1 et un CS-Spread $> 0,01$, le modèle de Heston représente généralement l'option la plus robuste. Dans ce cas, nous recommandons également de réduire la fenêtre de calibration pour éviter que les périodes de stress extrême ne biaisent excessivement l'estimation des paramètres, en particulier σ .

Ces correspondances entre signatures de liquidité et performance des modèles renforcent l'intérêt d'une approche intégrée, combinant l'analyse des proxies de liquidité de Sadefo Kamdem & Ayad (2021) et le framework MMFSV de Djetcha & Sadefo Kamdem (2024). Cette synergie méthodologique permet non seulement d'améliorer la sélection ex-ante des modèles, mais également d'optimiser le calibrage en fonction des régimes de liquidité observés.

En définitive, nos résultats démontrent que la "victoire" d'un modèle de valorisation n'est pas seulement déterminée par la classe d'actifs ou par la signature paramétrique (σ, ρ, α) , mais également par la structure de liquidité du marché sous-jacent, capturée par le quintuplet (RollSpread, CS-Spread, LIX, LogVol, ADV

A Annexe

A.1 Actions, tickers

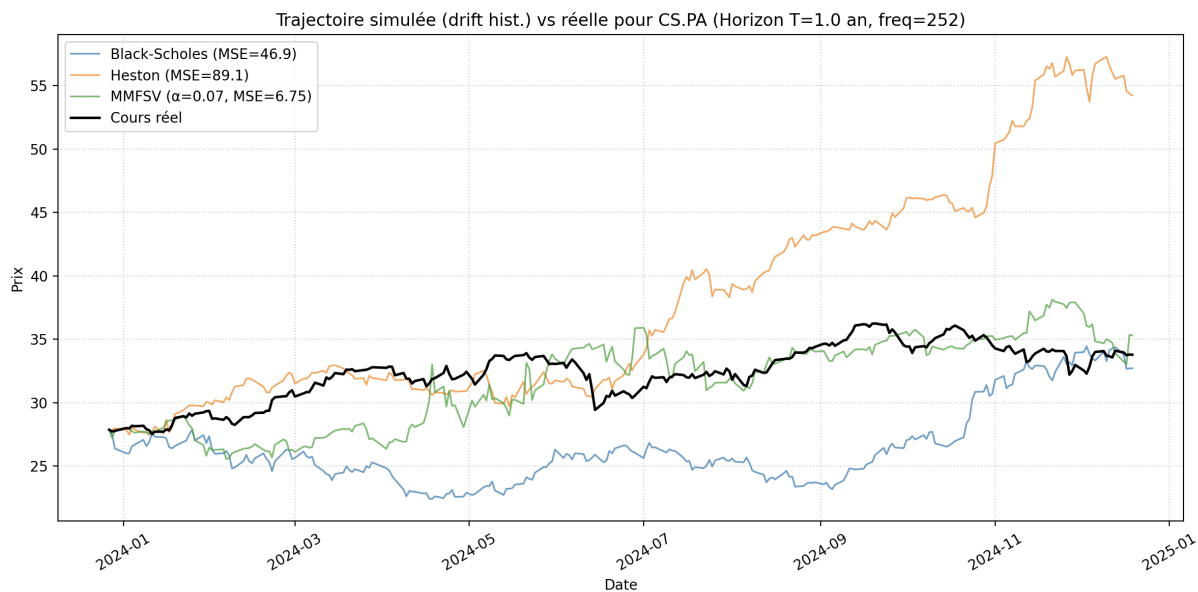


FIGURE 1 –

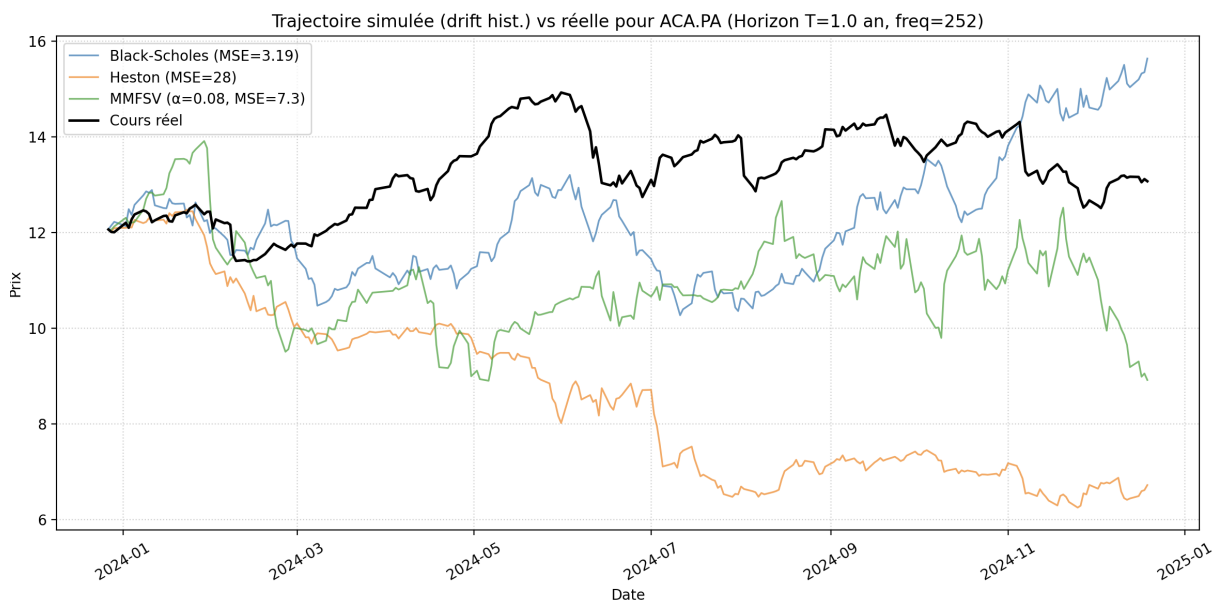


FIGURE 2 –

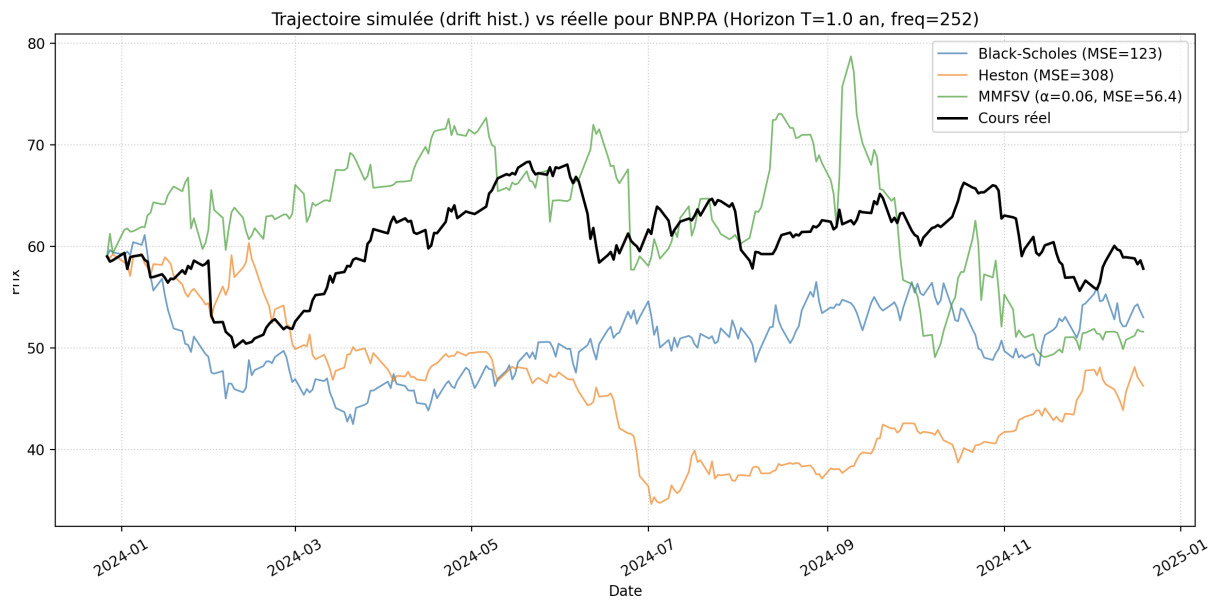


FIGURE 3 –

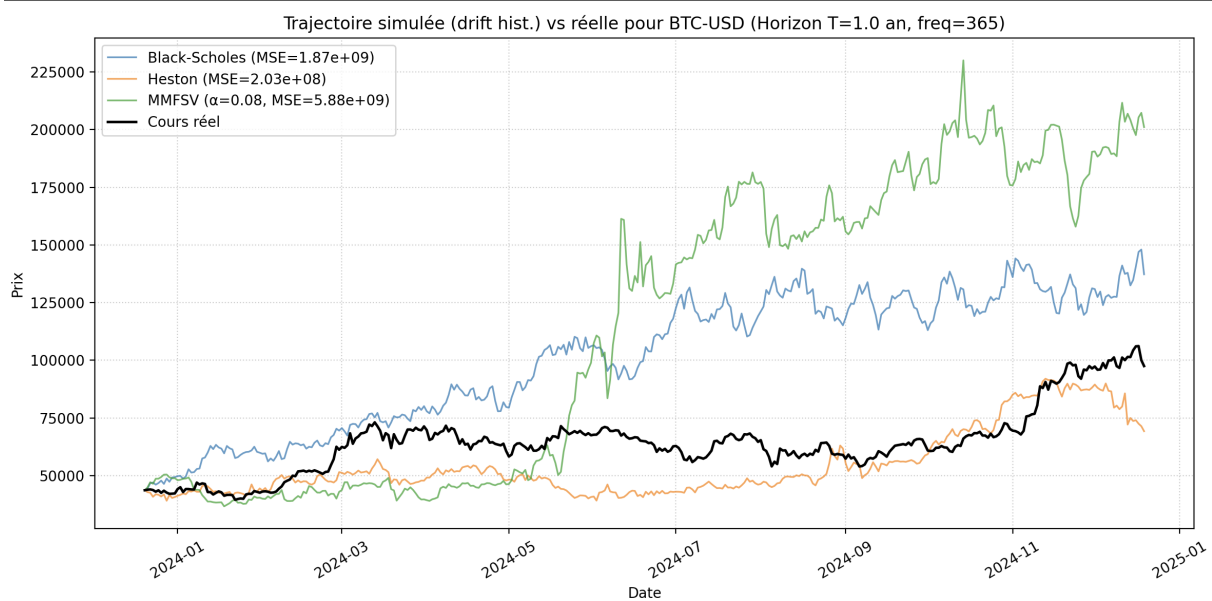


FIGURE 4 –

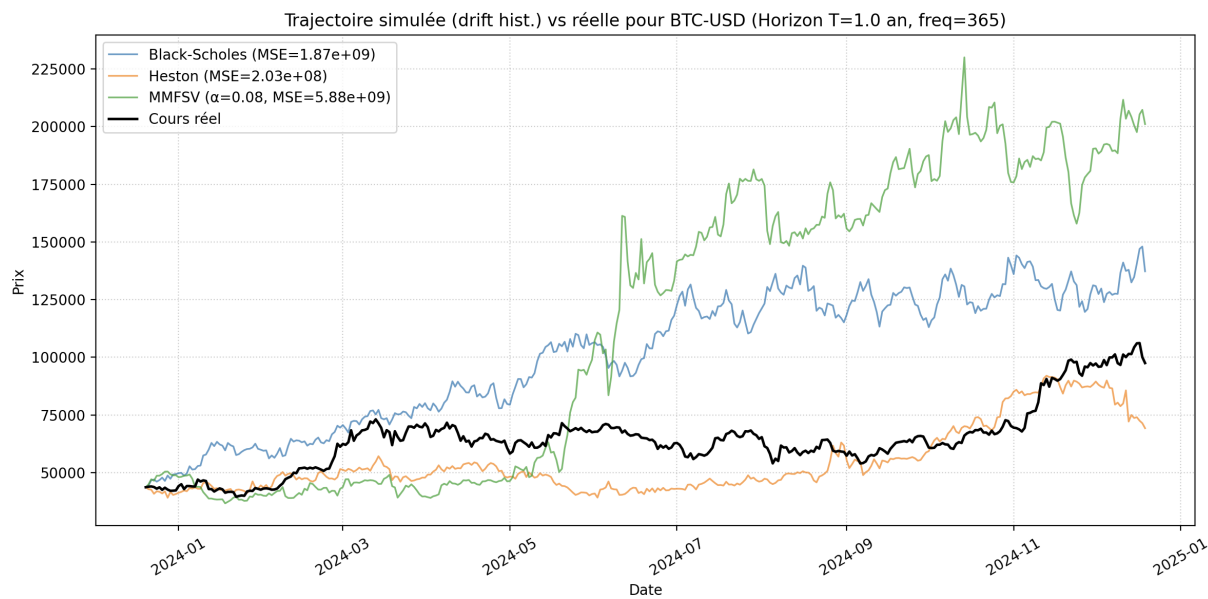


FIGURE 5 –



FIGURE 6 –

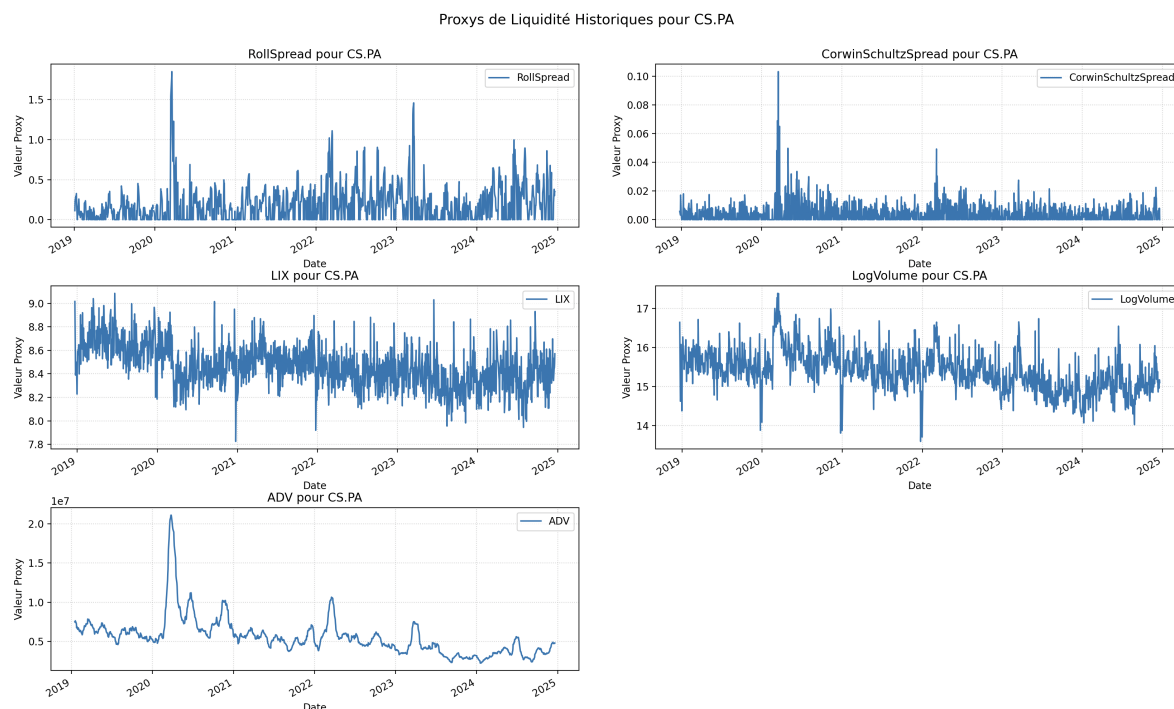


FIGURE 7 –

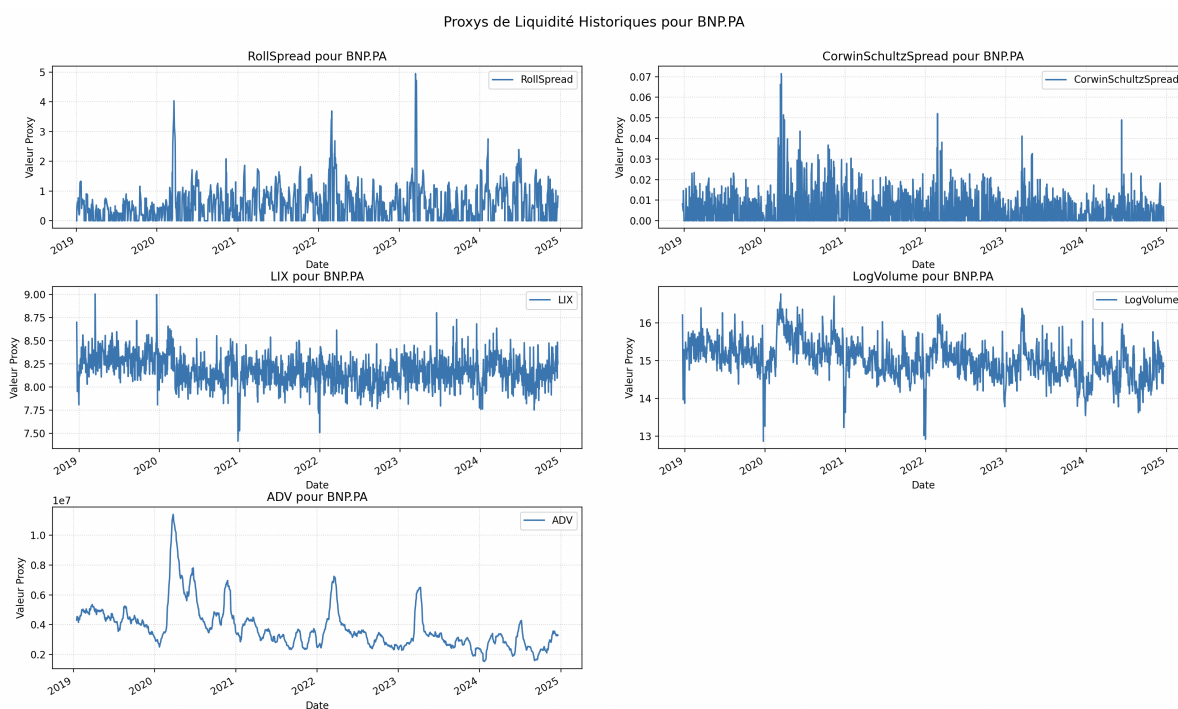


FIGURE 8 –

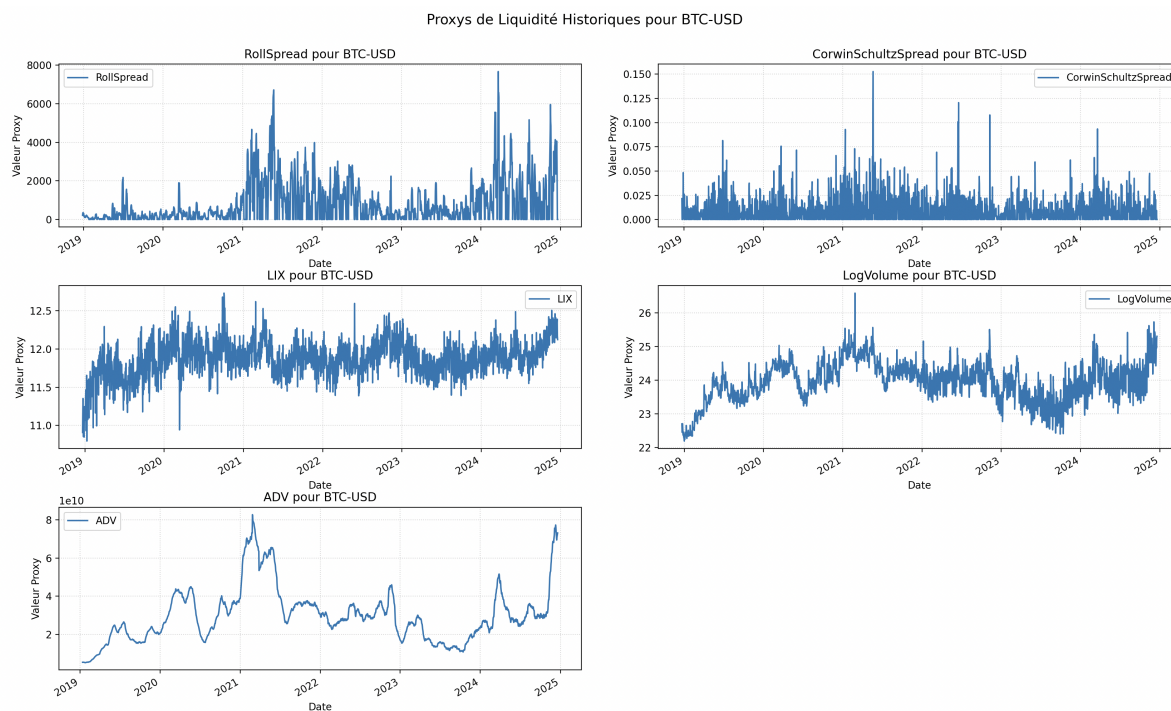


FIGURE 9 –

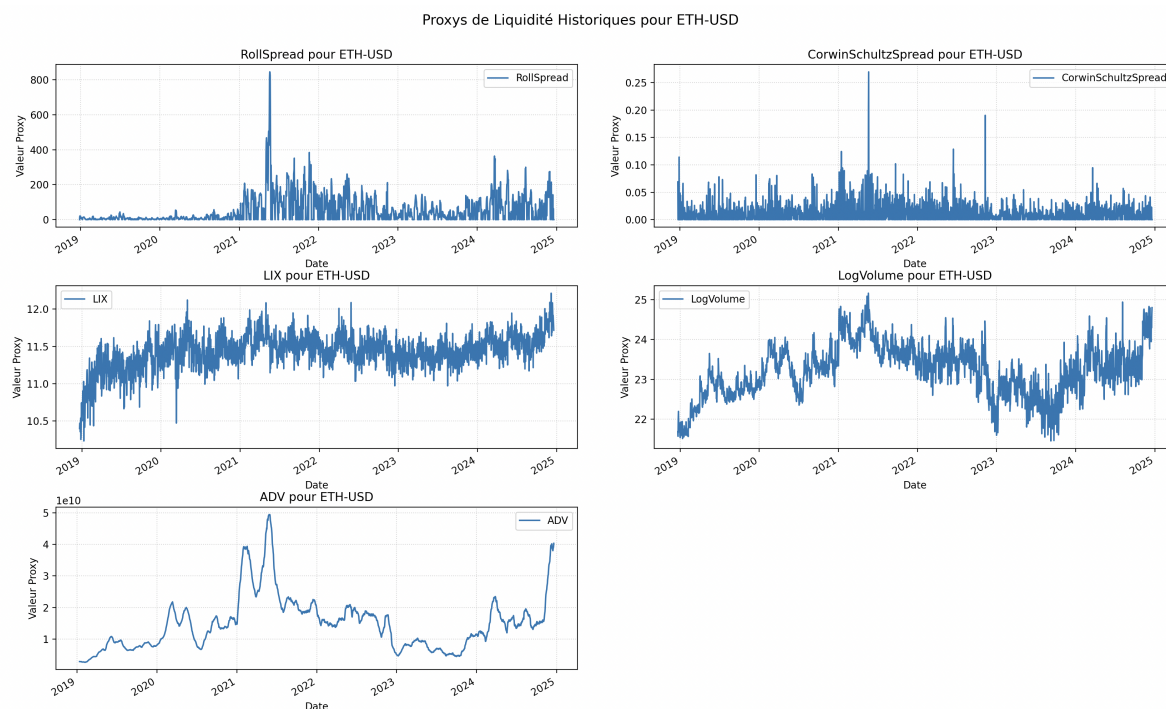


FIGURE 10 –

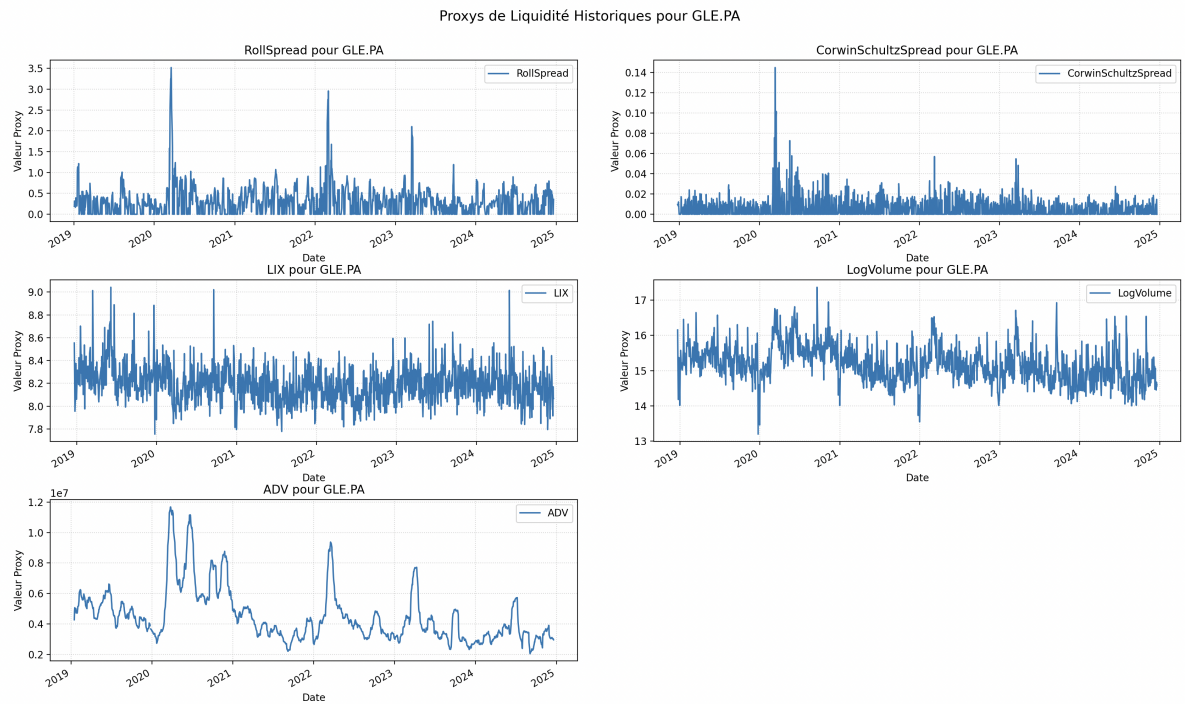


FIGURE 11 –